Portfolio Work Records

A source-grounded record of six products and research projects

Product journeys, technical architecture, methodology, measured results, contribution, and source maps

Prepared 14 October 2026

Contents

A portfolio source index and six detailed work records with supporting source maps. Page numbers exclude the cover.

Portfolio work records 4

Record index . . . 4
Required shape . . . 4
Evidence rules . . . 5
Editorial contract . . . 5

Relay - evidence-led employee service through conversation 6

Executive description . . . 6
The problem and the people affected . . . 6
What was implemented . . . 6
End-to-end journeys . . . 7
Architecture and important decisions . . . 7
Methodology: building an agent that can be improved . . . 8
Results and impact . . . 8
Why the work is notable . . . 9
Author role and attribution . . . 9
Bounded scope . . . 9
Evidence summary . . . 9
Exact source map . . . 10

Refract: session-centered research, recall, synthesis, and analysis 12

Product and evidence context . . . 12
The problem: research loses continuity when its evidence and decisions are split apart . . . 12
What was built: five workspaces over one research state . . . 12
Architecture and important implementation decisions . . . 13
Methodology: literature-led product design and testable instrumentation . . . 14
Results and why the work is notable . . . 14
Author role and attribution . . . 15
Evidence summary . . . 15
Source map . . . 16

Homeward - source-grounded hospital discharge assistance 17

Product outcome and Neerav's contribution . . . 17
The end-to-end workflow . . . 17
Architecture and methodology . . . 18
Why the work is notable . . . 19
Results and verified capabilities . . . 19

Source map . . . 20

Birb Hangout - a shared place for distant friends to spend time together 21

What was built and why . . . 21
The end-to-end experience . . . 21
Architecture and implementation decisions . . . 23
Creative-engineering methodology . . . 23
Results, significance, and beneficiaries . . . 24
Author role and attribution . . . 24
Evidence summary . . . 25
Exact source map . . . 25

Four-class Alzheimer MRI classification: CNN experimentation and probability ensembles 27

Study and evidence context . . . 27
The problem: make multi-stage MRI classification experimentation inspectable . . . 27
What was built: a complete notebook workflow over 6,400 images . . . 28
The ensemble contribution: combine model judgments at the probability level . . . 28
Methodology and evaluation provenance . . . 29
Why the work is notable . . . 30
Author role and attribution . . . 30
Evidence summary . . . 30
Source map . . . 31

Analyzing Brain Atrophy in Dementia Using Computer Vision 32

Publication and study context . . . 32
The human and research problem . . . 32
The implemented research contribution . . . 32
Dataset and study design . . . 33
End-to-end workflow . . . 33
Results and strongest evidence . . . 34
Why the work is technically notable . . . 35
Who benefits and what effort it reduces . . . 35
Authorship and role attribution . . . 36
Research lineage . . . 36
Evidence summary . . . 36
Exact source map . . . 37

Portfolio work records

These records provide detailed accounts of the problem, experience, implementation, methodology, results, contribution, and impact behind six portfolio works. Supporting source maps remain available for audit, while the main narrative is written for recruiters, hiring managers, engineers, and research reviewers.

Record index

Work	Record	Research depth	Primary evidence
Relay	`relay.md`	3,032 words	Current private source tree, evaluation snapshot, saved runs, tests, and Git history
Refract	`refract.md`	2,786 words	Current local source tree, research notes, evaluation records, tests, history, and thesis-positioning PDF
Homeward	`homeward.md`	2,318 words	Patient and caregiver workflow, React/Express/SQLite implementation, specialist contracts, deterministic controls, tests, and evaluation evidence
Birb Hangout	`birb-hangout.md`	3,501 words	Current private source tree, live About site, tests, change evidence, Git history, and 301-page research compendium
Four-class Alzheimer MRI classification	`alzheimers-mri-classification.md`	2,788 words	All 100 public notebook cells and retained outputs, README, six-commit history, and companion-result provenance
Brain atrophy paper	`brain-atrophy-paper.md`	3,210 words	Full five-page IEEE paper, seven figures, two tables, four equations, fourteen references, DOI metadata, and portfolio corroboration

Required shape

Each record should be approximately four to five prose pages (roughly 2,000-3,000 words) and should explain:

1. the human or organizational problem;
2. the implemented product or research contribution;
3. the end-to-end user workflow;
4. the architecture and important implementation decisions;
5. the methodology used to design, test, or evaluate the work;
6. the results and strongest defensible evidence;
7. why the work is technically or product-wise notable;
8. who benefits and what effort, risk, or fragmentation it reduces;

9 Neerav's direct contribution, while preserving factual publication co-authorship where applicable;

10 measured results, implemented capabilities, and the practical conclusion each result supports.

Evidence rules

1 Inspect the implementation, tests, evaluation artifacts, primary documentation, and commit context where available.

1 Prefer exact counts, named workflows, and observed results to broad adjectives.

1 Explain boundaries only where they clarify the scope of a positive claim; do not organize the record around missing work.

1 Do not convert a prototype into a production claim or a deterministic test into a live-model result.

1 End with a source map naming the files, URLs, commits, or paper sections that support the record.

Editorial contract

1 Start with the specific user outcome or research result. Do not begin with a technology list or a disclaimer.

1 Replace umbrella phrases such as "multiplayer play," "AI-powered," "research platform," or "improves workflows" with the concrete behavior, actor, object, and result documented in the relevant record.

1 Name the methodology that made the result credible: confirmation and state verification in Relay; one shared research session in Refract; deterministic authority and human review in Homeward; diegetic interaction and reproducible 3D review in Birb Hangout; probability-level ensemble comparison in the MRI notebook; explicit pixel masks and the `dem` ratio in the paper.

1 Pair every metric with its denominator, evidence source, and evaluation context. Do not detach a number from the run, split, artifact, or historical record that produced it.

1 Keep authorship precise. Present Homeward directly as Neerav's end-to-end work. Preserve the IEEE paper's factual co-authorship while stating Neerav's individual OpenCV measurement-pipeline contribution directly. Describe software and AI systems as tools or dependencies, not project contributors.

1 Keep the two Alzheimer projects separate. The IEEE paper reports a 66% to 71% crop comparison for an interpretable OpenCV atrophy proxy. The later notebook implements four-class TensorFlow CNN ensembles and has distinct result provenance.

1 Use scope boundaries as support for a strong exact claim, not as the headline. Never upgrade synthetic data, process-local services, deterministic tests, historical evidence, or a notebook experiment into production or clinical validation.

1 When a record distinguishes measured results, directly verified capability, and interpretation, preserve that distinction in the published sentence.

Relay - evidence-led employee service through conversation

Executive description

Relay is a voice-first employee workspace that turns fragmented employee-service work into one continuous, reviewable journey. Instead of asking an employee to know which portal, department, form, case queue, or calendar owns a problem, Relay begins with the employee's own question: "What do I still need to do?", "Open the form," "Why is my account blocked?", or "Set up time with IT." It reads the employee's current records, identifies the responsible party and next action, moves the existing portal to the relevant record, and prepares a bounded action for review. When a request changes state, the employee sees the exact proposal and explicitly confirms it before the application executes and verifies the result.

This is more than a conversational skin over a portal. The repository contains a React employee workspace, Fastify application boundary, typed synthetic company data, semantic navigation, five state-changing capabilities, confirmation and idempotency controls, post-write verification, text and voice adapters, and append-only evidence. It also contains the machinery to improve the agent: live and replay runners, held-out rewordings, grounding and tool scoring, safety and voice evaluation, statistical summaries, and a CI release gate.

Northstar Analytics is the fictional employer inside the product. Relay is the product and assistant brand. The records, cases, people, policies, and workflow effects are synthetic, which allows the work to demonstrate realistic employee-service behavior without using real employee data.

The problem and the people affected

Employee-service portals often expose the organization's internal boundaries rather than the employee's goal. A new employee may need to coordinate tax documents, direct deposit, training, equipment, account access, meetings, benefits, and support cases, but those records can live behind different navigation structures and be owned by different teams. The employee has to translate a plain-language need into system vocabulary, locate the right page, interpret status and ownership, and decide whether an action has actually happened. People Operations and service teams absorb the resulting follow-up: repeated status questions, incorrectly routed issues, incomplete tickets, duplicate work, and manual reconciliation.

Relay addresses that fragmentation while preserving the underlying systems' authority. Employees get a clearer path to completion; People Operations gets answers tied to current records and explicit ownership; service teams get better-scoped requests routed through a versioned catalog. Product and engineering teams gain a reusable workflow shell for identity, confirmation, idempotency, audit, and verification.

The design is useful where convenience and consequence meet. Natural language should simplify discovery without silently submitting a tax form, exposing private data, duplicating a case, or presenting a draft as a completed action. Relay combines conversational ease with application-owned control.

What was implemented

The employee-facing product is a complete synthetic portal rather than a chat-only demo. Its principal surfaces cover Home, My Onboarding, Tasks, Documents, Learning, Pay & Benefits, Help & Support, meetings, and workflow operations. Relay Guide opens beside this workspace on desktop and as a constrained mobile surface without replacing the portal. This matters because the employee can see the answer, destination, proposed action, and resulting record together.

Relay reads typed records across onboarding, tasks, documents, learning, equipment, meetings, cases, knowledge, pay, and benefits. It preserves status, owner, due health, blocker, employee action, and next-step owner. Missing, stale, conflicting, or unavailable required data produces a typed abstention rather than a model guess.

Navigation is semantic and application-controlled. The agent requests a registered route, panel, typed record, or allowlisted anchor, never an arbitrary selector or URL. The application maps the ID to the real route and DOM target, highlights it, and reports a visible postcondition. A follow-up such as "Open it" retains conversational reference while the portal decides what may be shown.

The write surface is deliberately small and concrete. Five registered capabilities are implemented:

1. `create_support_case` inspects existing cases and an approved route, prepares the request and duplicate check, then verifies that the created or reused case appears in the employee's collection.
1. `start_employee_task` checks ownership and the current state, transitions an eligible employee-owned task, and verifies that the same task re-reads as in progress.
1. `schedule_employee_reminder` validates the referenced task, document, or course and permitted delay, then schedules or safely reuses one process-local reminder.
1. `schedule_calendar_meeting` resolves an eligible attendee, shared hours and conflicts, then places the confirmed meeting on the Relay demo calendar with a stable ID.
1. `add_case_comment` checks that the case belongs to the employee and remains open, then adds the reviewed reply once to the same case activity.

Each capability supplies product-specific `inspect` and `execute` behavior. The shared workflow service supplies actor binding, confirmation, idempotency, ordered audit events, execution receipts, and post-write verification. This separation is a practical extensibility decision: adding a workflow requires a new adapter, while the safety and evidence lifecycle remains consistent.

End-to-end journeys

The flagship journey starts with "My VPN is failing." Relay retrieves the signed-in employee's context and checks related access records and cases. If timing or attempted fixes are missing, it asks one short question and re-reads context on the next turn. For a distinct issue, it opens Help & Support, selects IT Support, checks similar cases, and presents the exact proposal. No write has occurred. The employee reviews it and chooses Confirm or Cancel.

Confirmation sends an opaque token bound to the same actor and session. The server executes with a stable idempotency key, re-reads the cases, and reports success only when the returned case is present. The portal shows the stable ID, status, team, and next step. The audit progresses from inspection and reasoning through confirmation, tool execution, and `state_change_verified`; retries return the completed result instead of creating a second effect.

The same pattern supports "What's next?", exact-record navigation, reviewed case replies, reminders tied to the current record, and editable meeting drafts with conflict handling. Voice uses the same records, functions, workflow adapters, and confirmation UI as text, so speech is a transport into the product rather than a less-controlled implementation.

Architecture and important decisions

Relay is a typed TypeScript system with explicit browser, conversation, application, record, and evidence boundaries. React 19 and Vite render the portal and assistant; Fastify 5 owns HTTP and sessions. Shared contracts define domain records, navigation, functions, capabilities, reason codes, and evaluation snapshots. Ajv validates structured artifacts, and Vitest covers policy, orchestration, adapters, navigation, voice/text dispatch, secure input, persistence, and evaluation logic.

Deepgram's Agents SDK provides the real-time session, speech transport, playback, and interruption events. The evaluated configuration uses Nova-3, GPT-4.1, and Flux. Relay retains identity, data, navigation, policy, confirmation, mutation, and verification. The browser never supplies employee authority, and the long-lived Deepgram key stays server-side behind a short-lived client grant.

The foundational rule is "the model proposes; the application disposes." A function call to `propose_onboarding_action` is deferred until end of turn and can only produce a visible proposal. It cannot

perform the mutation. The later confirmation path is outside the conversational function call. This prevents a plausible-sounding response, a premature function request, or an interrupted turn from becoming authority to act.

Voice lifecycle is product state, not an opaque connection. The adapter separates greetings from answers, waits for playback to drain, retains one session across a multi-turn task, interrupts when the user starts speaking, and applies bounded reconnect and inactivity limits. These details make the workflow usable by voice rather than merely capable of producing audio.

Evidence is designed into the architecture. Runs preserve configuration and provenance; the UI runner checks routes, records, confirmations, calendars, tools, writes, and final state. CI replays recorded turns through the portal with live Deepgram disabled and fails if a socket opens. Live suites are separate and budget-capped.

Methodology: building an agent that can be improved

The evaluation program measures whether the application reached the right state, not whether the answer contained expected phrases. Seven multi-turn development workflows cover onboarding work, support, meetings, department routing, pay and benefits, learning and reminders, and service-team calendar coordination; a mobile scenario exercises the same interaction at 390 x 844. Five safety scenarios map to OWASP LLM risks including private-data disclosure, excessive agency, sensitive input, indirect prompt injection, and false claims after cancellation.

Each live attempt records revision, prompt variant, models, cost, tool calls, latency, checks, and failures. Success requires every turn to satisfy observable conditions. Fourteen held-out rewordings stay outside the fix loop; if one leads to a change, it is retired and replaced. The scorecard reports intervals, pass^k, tool recovery, latency, and cost.

Evaluation extends beyond task completion. Tool scoring checks whether record IDs came from the server and whether the expected record was reached. Grounding review checks atomic facts against tool results and plants errors to test the grader. A 100-reply sample measures answer completeness and usefulness with a judge calibrated to human labels. Voice runs measure outcomes, first-audio time, word error, key-term recall, and barge-in; matched audio compares key-term prompting off and on.

Early failures in navigation, dates, confirmation, sensitive input, record IDs, connection timeouts, voice highlighting, session limits, and rebooking became named fixes and reruns. CI runs tests, the committed gate, the build, and 764 replay turns without a paid session. The gate covers safety, faithfulness, task success, held-out behavior, tool use, answer quality, spoken turns, and barge-in.

Results and impact

The October 11, 2026 snapshot uses 37 source runs at revision `dab9b6a`. The broader saved evidence covers 252 live conversations, 764 live turns, 1,271 tool calls, 57 recorded voice sessions, and 764 replay turns.

After the fix pass, 103 of 110 development conversations completed correctly, or 93.6% with a Wilson 95% interval of 87.4%-96.9%. The earlier-period result was 26 of 37, or 70.3%; this before/after is explicitly reported as sequential periods rather than a controlled A/B. Held-out rewordings achieved 39 of 42, or 92.9% [81.0%, 97.5%]. The measured held-out difference from development was -2.1 percentage points with a range from -11.0 to +6.0, so the evidence found no measurable drop on the unseen phrasings.

Tool behavior became both accurate and recoverable. The agent reached the correct record on 186 of 187 scored turns. The first ID was correct on 158 of 187; 30 hallucinated IDs were all caught by validation, and 89 of 99 turns containing a tool error recovered in the same turn. This is a stronger result than merely suppressing bad model output: the application converts an imperfect model choice into a bounded error and gives the agent a path back to real records.

Voice evidence spans natural owner-recorded sessions and controlled workflows. Thirty-one of 34 requests were useful, including 24 fully right; the latest scripted set passed 24/24 turns and 7/7 workflows. First audio was 2.0 seconds median and 2.7 seconds p95; barge-in stopped playback at 51 ms p95. Key-term prompting moved recall from 101/108 to 106/108 and WER from 3.7% to 2.7% with 0/60 false insertions, but was not adopted because `p = 0.125` was inconclusive.

Grounding review checked 1,035 facts across 316 replies and found zero unsupported facts after reviewing five flags. Planted-error sensitivity was 92% for day-off dates, 90% for wrong people, and 65% for hour-off times. Answer quality passed on 87/100 replies; judge calibration reached 94% raw agreement and kappa = 0.83, with all 13 flags confirmed by the human reviewer.

The safety program recorded zero unsafe actions, proposals, forbidden calls, or unsafe reply violations in 21 runs. It also exposed an early weakness: pasted bank details reached tool arguments in 4/4 attempts. Input redaction moved that result to 0/5, with the original evidence retained.

The included usage sample cost $5.59, or $0.243 of Deepgram time per fully correct conversation. The snapshot satisfies every enforced gate. Median full turn time, 15.3 seconds, remains tracked against a 12-second goal, so the evidence supports correctness, recovery, safety, and voice interaction - not an across-the-board speed claim.

Why the work is notable

Relay's strongest achievement is the integration of product behavior and evaluation discipline. Many agent demos optimize for a convincing reply. Relay makes the portal's own state the judge, keeps the model outside the authority boundary, and records enough provenance to explain why a result passed or failed. Its navigation model replaces arbitrary browser control with semantic, site-issued targets. Its workflow model separates preparation from execution, binds confirmation to actor and session, makes retries idempotent, and verifies the resulting record. Its evaluation model combines live runs with a free replay suite, holds back unseen language, tests the grader, measures voice as interaction rather than transcription alone, and turns recurring checks into a release gate.

The practical improvement is reduced translation work. Employees can begin from their intent and remain in one workspace. Service teams receive a more complete, correctly routed request. Product teams get a pattern for adding actions without expanding the model's authority. Engineering teams gain reproducible evidence instead of relying on a hand-picked demo. Even where the prototype uses synthetic and process-local systems, the contracts make clear where a durable HRIS, case provider, calendar, or queue adapter would attach.

Author role and attribution

The reachable Git history in the inspected checkout contains 74 commits, all attributed to the `neerav-git` account. On repository evidence, Neerav's role spans product definition, employee-journey design, information architecture, TypeScript implementation across client/server/shared layers, synthetic data modelling, agent and tool-contract design, workflow safety, voice behavior, evaluation design and execution, statistical reporting, QA, documentation, and portfolio evidence. No separate human development team is named in the repository history, so this record does not invent team roles.

The work does build on external technology and evaluation services: Deepgram supplies the Agents SDK and speech infrastructure; GPT-4.1 is identified as the evaluated reasoning/function-selection model; Claude produced the saved LLM-judge labels; React, Fastify, Vite, Vitest, Ajv, and related open-source packages provide the application foundation. Those dependencies are credited as platforms and tools, not represented as co-authors of the product decisions and implementation contained in this repository.

Bounded scope

The demonstrated organization and employee data are fictional. The case system, meeting calendar, reminders, outbox events, workflow state, and audit storage are application-owned. Production HR, payroll, ticketing, calendar, email, and messaging providers connect through the capability seams but are not claimed as current integrations. The measured results describe the recorded scenarios, revisions, models, and run corpus rather than population-wide employee-productivity outcomes.

Evidence summary

Results with context

1. 103/110 post-fix development conversations completed correctly (93.6% [87.4%, 96.9%]); the earlier period was 26/37.
1. 39/42 held-out rewordings completed correctly (92.9% [81.0%, 97.5%]) with no measurable drop from development.
1. Correct record reached on 186/187 scored turns; all 30 invented IDs were caught by validation; 89/99 tool-error turns recovered.
1. 31/34 owner-recorded spoken requests were useful; the latest scripted set passed 24/24 turns and 7/7 workflows.
1. Recorded voice TTFA was 2.0 s median and 2.7 s p95; interruption-stop latency was 51 ms p95.
1. 0 unsupported facts remained after review of 1,035 checked facts; answer quality passed on 87/100 replies with kappa = 0.83 judge/human agreement.
1. Red-team runs produced 0 unsafe actions; typed sensitive-value leakage changed from 4/4 before redaction to 0/5 after.
1. Key-term prompting improved recall from 101/108 to 106/108 with no false insertions in 60 negative clips, but was not adopted because the result was not statistically conclusive.
1. The included usage sample reports $0.243 of Deepgram time per fully correct conversation.

Implemented capabilities

1. Server-resolved employee scope; browser-supplied identity and authority are rejected by design.
1. Typed record retrieval, abstention, semantic route/record/anchor navigation, and visible postconditions.
1. Five registered, confirmation-gated capabilities with inspect/execute separation, actor binding, idempotency, audit chronology, and state re-reading.
1. Text and voice share one application boundary; voice implements multi-turn continuity, playback drain, barge-in, reconnect, and session limits.
1. Outcome-based live runner, no-cost replay runner, held-out suite, safety suite, grounding and tool scoring, calibrated judge, key-term experiment, snapshot generation, and CI release gate.

What the evidence demonstrates

1. Relay reduces employee-service fragmentation by letting an employee begin with intent while keeping the record and action visible in the same workspace.
1. The architecture is reusable because product-specific adapters plug into a shared confirmation, idempotency, evidence, and verification shell.
1. The evaluation system is a substantive product-engineering contribution: it found concrete failures, supported targeted fixes, and protects measured behavior in CI.
1. The work demonstrates a credible pattern for safe application-native agents without claiming that synthetic adapters are production enterprise integrations.

Exact source map

1. `README.md` - product definition, one-minute workflow, five capabilities, architecture diagrams, integration boundaries, testing/evidence levels, and reproduction steps.
1. `package.json` - React/Fastify/Deepgram/Ajv/Vitest stack and build, test, dataset, live-evaluation, scoring, snapshot, and gate commands.
1. `src/server/app.ts` - HTTP/session boundary, token grant, identity-field controls, context, navigation, workflow, confirmation, and evaluation endpoints.
1. `src/server/intelligence-guide.ts` and `src/server/synthetic-company-store.ts` - employee-scoped context, record index/detail resolution, typed source conditions, and semantic navigation inputs.

1. `src/server/onboarding-workflow.ts`, `src/server/workflow-capabilities.ts`, and `src/shared/workflow-capabilities.ts` - preparation, confirmation-token binding, idempotency, capability registry, execution, verification, receipts, and audit events.
1. `src/client/DeepgramTextGuide.ts` and `src/client/voice/DeepgramVoiceAdapter.ts` - prompt/function dispatch and the text/voice session lifecycle.
1. `evaluation/ui-runner/scenarios.ts` - W1-W7, M1, S1-S5, held-out phrasing rules, and observable route/state/safety expectations.
1. `evaluation/scorecard/scorecard.ts` and `evaluation/scorecard/metrics.ts` - task outcomes, pass^k, Wilson intervals, stratified comparisons, tool recovery, latency, safety, and cost aggregation.
1. `evaluation/summaries/evaluation-snapshot.json` - frozen October 11 metric snapshot at revision `dab9b6a`, including denominators, intervals, methods, fixes, scale, voice, grounding, safety, and answer quality.
1. `docs/EVALUATION_CASE_STUDY.md` and `docs/EVALUATION_SPEC.md` - concise method/results narrative and the broader metric, reporting, voice, safety, and change-control contracts.
1. `evaluation/gate.ts` and `.github/workflows/ci.yml` - enforced evaluation thresholds and the no-live-session CI sequence.
1. `evaluation/judge/`, `evaluation/grounding/`, `evaluation/tool-args/`, and `evaluation/keyterms/` - answer-quality calibration, factual support checking, tool-argument accuracy, and matched audio experiments.
1. `docs/AGENTIC_WORKFLOW_TRACE.md`, `docs/SYSTEM_MODEL.md`, and `docs/PHASE1_AUTOMATION_FOUNDATION.md` - workflow chronology, state ownership, trust boundaries, and secure-field design.
1. `docs/assets/readme/relay/CAPTURE_MANIFEST.md` and its seven screenshots - versioned visual proof for the portal, grounded answer, focused record, prepared and confirmed case, calendar, and evidence trace.
1. Commits `dab9b6a` (CI evaluation gate), `7a39ed6` (24/24 spoken turns and 7/7 workflows), `34aaa71`/`9b5b4df` (calibrated answer-quality evidence), and current inspected head `394ea86` (evaluation page/case-study release-gate wording).
1. External evaluation dashboard linked by the repository: https://claude.ai/artifact/2vMEzrimhQtoprShg4uukV

Refract: session-centered research, recall, synthesis, and analysis

Product and evidence context

Refract is a research workspace that joins activities researchers normally perform in separate tools: close reading of papers, source-linked note capture, cross-paper synthesis, retrieval practice, dataset inspection, and statistical modeling. The public repository and retained product evidence cover the React interface, FastAPI routes, SQLAlchemy models, retrieval, grading, session, comparison, and statistical-analysis services; migrations; browser probes; evaluation reports; research notes; and the thesis-positioning document.

The problem: research loses continuity when its evidence and decisions are split apart

The human problem is not simply that reading papers is slow. It is that a research argument is assembled across disconnected environments. A researcher reads in a PDF application, copies excerpts into notes, asks questions in an AI chat, compares papers in a document, practices recall elsewhere, uploads data to a notebook, and finally writes an interpretation that must somehow reconnect model output to the literature. Each move sheds context. Notes drift away from page references; comparisons become prose without a recoverable source set; review cards lose the passage that made them meaningful; and statistical results are discussed after the fact rather than interpreted through the research question and papers that motivated them.

Refract's main contribution is a continuity model for this work. A research session is not a folder label. It is a persistent object connecting a title, topic, context, papers, memberships, saved passages, question-answer cards, review attempts, datasets, analysis runs, and comparison output. The same session framing can shape document chat, determine the source set for comparison, scope a dataset, and provide literature evidence for interpreting model results. This converts a collection of useful features into one traceable research workflow.

The audience is broader than a student who wants summaries. Graduate researchers, literature-review teams, and applied analysts benefit when they must defend where a claim came from, why a variable entered a model, what evidence was available at the time, or whether a learner recalled rather than reread. In Industrial and Systems Engineering, the Battery Remaining Useful Life demonstration makes the value concrete: the researcher can study degradation literature, collect evidence, compare methods, audit a battery dataset, run baseline models, and interpret the resulting signals without severing the connection between the qualitative and quantitative work.

What was built: five workspaces over one research state

The product exposes five distinct workspaces rather than treating every task as chat.

Reader keeps the PDF, session navigation, and assistant available together. A user can upload a PDF, navigate a generated or native table of contents, select a passage, ask for an explanation, simplification, key terms, a quiz, or a summary, and save the passage and response. Page-aware source badges return the user to the document. The viewer uses windowed page rendering, most-visible-page tracking, out-of-window navigation retries, and persistent selection highlighting so the research object remains present while the assistant is used.

Index turns those interactions into durable research memory. Saved entries retain page location, passage text, deep section path, concept tags, rhetorical facet, notes, synthesis, question origin, source chunk identifiers, and review state. The interface can group by section, concept, origin, or review pressure. It also contains repair paths for older cards whose standalone question or source context is weak. A semantic duplicate gate compares a proposed study question with active cards on the same highlight using normalized MiniLM embeddings and a cosine threshold of 0.85; the user can open the existing card, merge an answer, force-save, or dismiss. Duplicate cleanup is a soft-delete workflow so historical review rows remain joinable for research.

Review closes the loop that most reading assistants leave open. A card presents a canonical study question, asks the learner to answer from memory, captures a 1-5 confidence rating before grading, and records recall latency. The default is free recall with the source hidden. Where appropriate, the learner can reveal a cloze or assisted passage, and the system records `reveal_used` and `recall_mode` so assisted attempts are not silently mixed with free-recall data. Claude grades the answer against the source passage and expected answer on three 1-5 dimensions: core claim, supporting detail, and faithfulness. The combined score maps to an FSRS rating, updates stability, difficulty, state, and next due date, and stores both the raw grading result and derived scheduler input. The product therefore automates a sequence that would otherwise require manually creating cards, self-scoring recall, and updating a schedule.

Compare creates a persistent cross-paper research artifact. It combines a deterministic baseline with a generated analysis path. The generated path reads sampled PDF chunks first and treats saved notes as secondary anchors; it produces per-paper analyses, fixed scholarly dimensions, session-specific comparison axes, matrix cells, cross-paper insights, research gaps, and reader-guidance cards. Session title, topic, and context shape the comparison. Cache invalidation detects changes to the session framing or paper membership, and asynchronous generation records stage, progress, heartbeat, error, and stale state. This is notably more structured than asking a model to "compare these PDFs" and receiving disposable prose: the implementation makes the question, source set, dimensions, contrasts, gaps, and generation state inspectable.

Stats extends the same session into quantitative work. A dataset is uploaded as a session-scoped artifact and its schema, shape, and storage identity are persisted. The current staged run contains setup, dataset profile, initial audit, and baseline regression checkpoints. The profile reports row and column counts, duplicates, missingness, inferred kinds, numeric summaries, target candidates, role hints, and variable semantics. The audit checks missingness, constants, identifier-like variables, target correlation, probable leakage at `|Spearman rho| >= 0.97`, and predictor redundancy at `|rho| >= 0.95`. Regression is gated on human acceptance of the audit rather than firing automatically. It then compares a train-mean baseline, standardized linear regression, and a Random Forest regressor using MAE, RMSE, and R-squared. The active code uses an 80/20 split with `random_state=42`; the forest uses 240 trees, maximum depth 14, minimum leaf size 2, median imputation, and single-process execution. Predictor exclusions, step versions, acceptance, stale status, assumptions, warnings, metrics, coefficients, feature importances, artifact manifests, and review decisions are retained with the run.

Architecture and important implementation decisions

The frontend is React 18 with Vite, Zustand, Axios, React Markdown, and `react-pdf`. The backend is FastAPI with SQLAlchemy, Alembic, PostgreSQL, Pydantic settings, and Uvicorn. S3-compatible object storage holds uploaded files and generated artifacts. ChromaDB holds document vectors. Analytics use pandas, NumPy, scikit-learn, Matplotlib, Seaborn, and OpenPyXL. Claude is called directly for document response generation, comparative synthesis, ontology/classification, card generation, and recall grading; OpenAI is used for Whisper transcription. The code does not hide the product behind a generic orchestration framework: the important domain logic is visible in custom services, typed routes, stored state, and explicit prompts.

The retrieval path is concrete. PyMuPDF extracts text page by page. `tiktoken` with `cl100k_base` makes 512-token chunks with 50-token overlap. `all-MiniLM-L6-v2` produces L2-normalized embeddings, and ChromaDB stores them in the cosine-distance `pdf_chunks` collection with `pdf_id`, page number, and chunk index. A document question retrieves the top five chunks filtered to the active PDF. When all retrieved distances exceed 1.3, highlighted questions remain bound to the selected passage, obvious document questions stay on the document path, and genuinely external questions can use a separately labeled Tavily path. Responses return page and chunk metadata to the UI. This supports the bounded positive claim that Refract implements source-constrained retrieval with page-level provenance and evidence-only fallback paths. It is not described here as an independent claim-by-claim citation verifier.

Several design decisions show methodological discipline. Session suggestions are deterministic, editable, and safe to ignore rather than silently model-generated. PDFs can belong to more than one session through a membership table while retaining a primary-session compatibility pointer. Comparisons become stale when their source set or framing changes. Statistical runs require review and acceptance between audit and modeling steps. Card deletion is archival

where research joins must survive. Rubric version, prompt version, exact model string, source-reveal mode, raw scores, and scheduler transitions are recorded because changing any of them changes the meaning of later analysis.

The author also designed recovery and legacy-repair paths instead of assuming a clean greenfield database. Backfill routes establish default sessions, older question cards can be reframed or converted to notes, source context can be reattached, duplicate cards can be merged without deleting research history, and migrations provide forward and downgrade paths. These choices reduce the practical risk of evolving a research prototype while preserving longitudinal records.

Methodology: literature-led product design and testable instrumentation

The project was developed as a system-building thesis with a deliberately narrower empirical arm. The research folder surveys spaced repetition and FSRS, testing-effect evidence, LLM grading, pedagogical language models, scholarly-reading interfaces, confidence calibration, and authentic classroom studies. Its gap analysis distinguishes components that already exist from the specific integration being investigated. The strongest thesis idea is not that RAG, FSRS, or LLM grading is individually new. It is the capture-to-schedule-to-test-to-grade-to-reschedule loop over user-selected academic passages, with source-grounded grading and analyzable review records.

That literature review directly shaped the product. Testing-effect and generative-learning findings motivated answering before reveal. Calibration research motivated collecting confidence before the grade is shown. FSRS provides the scheduling model, but the scheduler signal comes from a structured assessment rather than the learner's unexamined "again/hard/good/easy" judgment. Work on scholarly reading motivated rhetorical facets, passage-level evidence, and cross-paper comparison. Cautionary work on learning versus immediate performance led the research plan to separate implemented instrumentation from a future claim about delayed retention.

The grading design is intentionally reproducible. The active service pins `claude-sonnet-4-20250514`, sets temperature to zero, uses rubric version `v1.0` and prompt version `grading_v1.0`, passes source chunks and an expected answer, requests JSON-only output, and stores the model's confidence, rubric hits, missing concepts, and feedback. The 1-5 overall result is preserved alongside the 1-4 FSRS conversion so researchers can revise the mapping after the fact without rerunning model grading. A `human_grades` table and blinded export path support an inter-rater-reliability substudy rather than assuming that model judgment is ground truth.

Evaluation is layered. Backend tests cover health, PDF upload and streaming, table-of-contents shape, chat with selection and history, source schema, and error cases. Playwright probes cover session creation and editing, sidebar placement, workspace navigation, index/review behavior, comparison payloads, legacy repair, recall-first interaction, duplicate handling, and Stats layout. Stored historical reports show a nine-of-nine UI evaluation for loading, PDF selection, formatted chat, five visible source badges, Index navigation, adaptive concision, and 1440/1024 viewport overflow; a one-of-one multi-turn concision case also passed. The deep-fix status record reports six-of-six duplicate-flow checks and eight-of-eight recall-first checks. These are useful verification results, not user-outcome evidence, and they are dated artifacts rather than a fresh test run performed for this record.

Results and why the work is notable

The strongest achieved result is an unusually complete traceability chain. A user can move from a paper page to a selected excerpt, a grounded response, an indexed card, a recall attempt, a rubric result, and a new review date without losing the document and chunk identifiers that anchor the sequence. On the quantitative side, the user can move from a session goal and dataset to a profile, a leakage/redundancy audit, an accepted feature frame, three model comparisons, saved metrics, and literature-alignment notes. Refract therefore automates both research-memory maintenance and a first-pass analytical audit while preserving human checkpoints.

The system is also notable for making provenance visible at several levels rather than adding citations cosmetically at the end. Document answers return pages and chunks. Comparison output is fingerprinted to its framing and paper set. Recall grades retain source text, prompt/rubric/model versions, reveal behavior, and scheduler transitions. Statistical variable rows distinguish data-derived evidence, domain or preset knowledge, literature support, and unresolved meaning. These structures let a later reviewer ask not only "what did the model say?" but "which source, version, state, and decision produced this result?"

The eight-page thesis-positioning document frames Battery Remaining Useful Life as the demonstration case and cites prior notebook metrics of RMSE 25.61 and R-squared 0.993. Those values are evidence from the author's earlier validated analysis, not a newly measured Refract run in this review. The directly verified Refract result is the implemented, review-gated pipeline that computes its own mean, linear, and forest metrics and retains them with exact inputs and assumptions. That is already meaningful: it replaces an opaque "run model" button with a staged notebook-like record that makes dataset quality, leakage risk, predictor decisions, baseline value, and interpretation inspectable.

For researchers, the improvement is less reconstruction work and fewer undocumented transitions. For learners, it is a move from passive highlighting to source-grounded recall practice with feedback and scheduling. For advisors or reviewers, it is a clearer audit trail across literature and analysis. For an organization conducting applied research, it offers a pattern for keeping qualitative evidence, quantitative evidence, and the decisions joining them in one recoverable system.

Author role and attribution

Neerav designed the research model and product workflow and built the end-to-end system: session model, document reader, evidence index, comparison workflow, active-recall pipeline, research exports, migration and repair paths, and statistical workspace. He also translated the literature review into testable product instrumentation and built the browser and backend evidence used to verify the workflow.

Evidence summary

Results with context

1. Historical stored UI evaluation: 9/9 checks passed, including formatted chat, five visible page-source badges, Index switching, adaptive concision, and no horizontal overflow at 1440 and 1024 pixels.
1. Historical stored multi-turn evaluation: 1/1 concision scenario passed; the follow-up response was shorter while retaining the required structure.
1. Deep-fix verification records: duplicate workflow 6/6 and recall-first workflow 8/8.
1. Thesis-positioning evidence reports an earlier Battery RUL notebook result of RMSE 25.61 and R-squared 0.993. This is a prior benchmark, not a freshly reproduced Refract result in this inspection.

Implemented capabilities

1. Five implemented workspaces share a persisted research-session layer: Reader, Index, Compare, Review, and Stats.
1. PDF ingestion, 512-token/50-overlap chunking, MiniLM embeddings, per-PDF top-five cosine retrieval, page-linked sources, and threshold-based fallback routing are implemented in active services.
1. Free-recall submission, pre-grade confidence, a three-dimensional source-grounded rubric, FSRS state updates, model/prompt/rubric logging, assisted-reveal telemetry, human-grade storage, and research export are implemented.
1. Session-scoped dataset upload, profiling, leakage and redundancy audit, explicit step acceptance, mean/linear/Random Forest comparison, MAE/RMSE/R-squared output, feature-importance and coefficient records, visual artifact manifests, and review decisions are implemented.
1. Cross-paper comparison records source-set and framing fingerprints, asynchronous progress, stale/interrupted states, per-paper analyses, comparison axes, analytical insights, gaps, and guidance.

What the evidence demonstrates

1. Refract's most defensible differentiator is not a single algorithm; it is the session-centered integration of research synthesis, active recall, and review-gated quantitative analysis with recoverable provenance.

1. The implementation reduces tool-switching and manual reconstruction because the artifact relationships are stored, not merely described in UI copy.
1. The instrumentation is strong enough to support a future retention study and human-grader agreement study. It does not by itself establish improved long-term retention or production predictive-maintenance performance.

Source map

1. Product overview and current architecture: `/Users/neeravch/Desktop/pdf-workspace/README.md`; public repository `https://github.com/neerav-git/Refract`.
1. Workspace UI and end-to-end behavior: `frontend/src/App.jsx`; `frontend/src/components/PDFViewer.jsx`; `ChatPanel.jsx`; `WorkspaceIndex.jsx`; `WorkspaceReview.jsx`; `WorkspaceCompare.jsx`; `WorkspaceStats.jsx`; `ReviewSession.jsx`; `frontend/src/store/index.js`.
1. Sessions and source membership: `backend/app/models/research_session.py`; `backend/app/services/research_session_service.py`; `backend/app/routers/research_sessions.py`.
1. Retrieval and grounding: `backend/app/services/pdf_service.py`; `embedding_service.py`; `chroma_service.py`; `chat_service.py`; `frontend/src/utils/linkifyPages.jsx`.
1. Recall grading and research instrumentation: `backend/app/services/grading_service.py`; `backend/app/routers/review.py`; `backend/app/models/review.py`; `backend/app/routers/research.py`; migrations under `backend/migrations/versions/`.
1. Comparative synthesis: `backend/app/services/comparative_analysis_service.py`; `frontend/src/components/WorkspaceCompare.jsx`; `session_prompt_and_research_flow_report.md` and its five-page PDF counterpart.
1. Statistical workflow: `backend/app/models/analysis_run.py`; `analysis_step.py`; `session_data_file.py`; `backend/app/routers/analysis.py`; `data_files.py`; `backend/app/services/analysis_execution_service.py`; `analysis_models_service.py`; `analysis_evidence_service.py`; `analysis_semantics_service.py`; `analysis_visualization_service.py`; `frontend/src/components/WorkspaceStats.jsx`.
1. Research basis and methodology: `research/00_README.md`; `01_literature_landscape.md`; `02_gap_analysis.md`; `03_methodology_reference.md`; `04_build_decisions.md`; `VISION.md`; `docs/deep_fix_status_2026-04-22.txt`; `docs/industrial_systems_research_layer_plan.md`.
1. Thesis framing reviewed in full: `07_thesis_positioning.pdf`, especially Parts 1-4 (problem, qualitative contribution, quantitative contribution, and overall architecture).
1. Evaluation evidence: `backend/tests/`; `backend/tests/eval/test_cases.json`; `backend/tests/eval/last_report.json`; `backend/tests/eval/last_ui_report.json`; browser probes under `playwright-tests/`, especially `step2_dedup_verify.js`, `step3_recall_first_verify.js`, `part4_legacy_context_repair.js`, `workspace_compare_part3.js`, and `stats_notebook_probe.mjs`.
1. Commit context: `4304a61`, `e9a7c8e`, `df85518`, `9f7c58a`, `0977376`, `73a1351`, `51bc929`, `1c0c443`, `00ba53e`, `9439812`, `9fa3f06`, and `003cac4`.

Homeward - source-grounded hospital discharge assistance

Product outcome and Neerav's contribution

Homeward is a patient-facing discharge follow-through system that turns a hospital record into a persistent, source-linked recovery plan. It helps a patient identify the next recorded step, keeps missing or conflicting information visible, lets a local care-team operator review uncertain instructions, and makes reminder creation an explicit, auditable, safely retryable action.

Neerav conceived and built Homeward end to end: the product direction, patient journey, interface, backend, persistence, synthetic patient data and Synthea integration, source-grounded specialist architecture, deterministic workflow controls, evaluation, and technical documentation. He is continuing its development toward launch.

The core problem is the handoff between receiving discharge paperwork and acting on it at home. A patient may need to arrange follow-up, bring documents, complete a check-in, clarify a laboratory instruction, and remember dates, while also distinguishing the hospital's actual directions from historical records or general educational material. That is an information-coordination problem with real consequences: unsupported certainty is dangerous, but a product that only warns and refuses is not useful. Homeward's answer is to make the record operational without silently upgrading unreviewed text into clinical truth.

The implemented experience supports three hand-authored fictional patients plus one patient derived from an official Synthea sample. In the primary Alex Morgan scenario, the plan exposes four concrete tasks: arrange a follow-up visit, gather paperwork, complete a dated recovery check-in, and clarify a laboratory instruction whose test and date are absent. The application does not fill those gaps. It carries the clarification state through the patient workspace, source search, summary, help flow, care-team view, and reminder eligibility rules. This is a strong product decision: uncertainty is modeled as durable state rather than softened into vague prose.

The end-to-end workflow

A patient begins in **My recovery**, where Homeward presents a deterministic next step derived from the current plan. The selector considers only pending, unpaused tasks with valid source references and valid recorded dates. It orders eligible items by recorded date and explicitly describes that order as scheduling, not medical urgency. Completed, rejected, unreviewed, help-paused, and clarification items remain visible in their own states rather than contaminating the recommendation. The dashboard and the question "What is my next step?" consume the same guidance object, so the card and conversation cannot drift into two competing interpretations of the plan.

Every task retains an exact source reference: document ID, section ID, and quote. The patient can inspect the underlying passage instead of trusting a generated summary. Date provenance is also represented. A date may come from a reviewed source, a human clarification, or a legacy plan entry, and Homeward labels that distinction. Relative phrases are not converted into dates unless the workflow has acceptable evidence. An unknown deadline remains `null`, which prevents an apparently convenient reminder from laundering ambiguity into a scheduled action.

The **Ask Homeward** workspace offers source search and a TrueForge-backed agent path. Recognized plan-navigation questions always remain deterministic, even if the user selected live mode. Source search retrieves exact passages lexically from the patient's own small corpus without requiring embeddings or a vector database. A saved-record summary groups ready tasks, items needing attention, reported completion, history, imported passages, and historical context. This gives the patient a usable explanation even when no model is configured.

When live execution is available, a deterministic router sends one request to one of four specialists: a care-plan coordinator, discharge summarizer, questions-and-sources specialist, or reminder assistant. These are not cosmetic labels over a universal agent. Each role has its own native AgentSpec and patient-and-role-scoped MCP endpoint, and the server enforces a different tool inventory. The summarizer can read summaries and search sources but cannot

propose or execute reminders. The reminders role can inspect the plan, propose a reminder, and execute only an already approved reminder. No role receives instruction approval, help resolution, medication modification, appointment booking, outbound messaging, or emergency-dispatch tools.

The model is deliberately denied authority over clinical wording. Its structured output selects source, task, and action identifiers and, optionally, one allow-listed education topic. The backend verifies those references against the current patient snapshot and reconstructs the answer from stored records. Foreign, fabricated, malformed, or stale references are rejected. If the plan changes while a model turn is running, or if the output cannot be verified, the app returns a labeled saved-record fallback. This transforms the language model from an unbounded answer generator into a constrained reference selector inside a deterministic product workflow.

Unclear information has a separate human path. A patient can request help on a task, which persists in SQLite and appears in the local care-team workspace. An open help request pauses completion, reminder proposals, and execution of a previously approved reminder. The operator can respond or resolve the conversation, but doing so does not approve the instruction or invent a deadline. Review decisions preserve the original citation and create immutable audit history containing the prior and resulting task snapshots, source references, request ID, and locally assigned actor category. Optimistic revisions prevent one reviewer from overwriting a newer decision made in another session.

Document import follows the same discipline. Homeward accepts pasted text, TXT/Markdown, and text-based PDFs, splits the content into searchable passages, and keeps them unreviewed. A user or agent may propose an exact imported passage for review; the system does not infer a deadline or make it actionable. Review can confirm, clarify, or reject it. Historical Synthea content is explicitly segregated from discharge orders, with provenance down to source file, row, fields, and hash. This lets the product demonstrate realistic longitudinal context without pretending that a medication history establishes a current discharge regimen.

The reminder workflow shows the same attention to authority and failure recovery. A known, dated task can produce a proposal. A separate Homeward UI/API step approves or declines it; there is intentionally no approval MCP tool. Execution creates a local `.ics` calendar file - not a booked appointment, sent text, or background notification. The demo can simulate a response being lost after the reminder was stored. Retrying returns the original receipt instead of creating a duplicate. Action fingerprints bind reminders to a task revision, instruction, source, and date, so later care-plan changes invalidate stale approvals even if a superficial status happens to look unchanged.

The persistent US-demo emergency control is also independent of agent readiness: it is a native `tel:911` link initiated by the user's device, not a claim that an AI dispatched help. General education is similarly separated from patient-specific direction. The server performs fixed-topic MedlinePlus lookups, restricts returned links to the MedlinePlus domain, caches successful results for twelve hours, falls back to curated links, and sends no patient identifiers. Education can inform a conversation, but it cannot modify the care plan.

Architecture and methodology

Homeward is a React 19 interface backed by an Express 5 API and SQLite persistence. The store contains patients, documents, tasks, actions, audit events, idempotency receipts, and operational traces. TrueForge provides the optional session/turn runtime and calls patient-bound MCP tools. This separation is the architectural thesis: probabilistic components retrieve, select, and explain, while deterministic services own task state, approval, eligibility, dates, receipts, and recovery from failure.

The implementation method was contract-first and evidence-led. Neerav defined explicit contracts for discharge review, next-step selection, source trust, role capabilities, and live evaluation. Shared scenario functions are reused by domain tests and the evaluation runner, reducing the chance that presentation evidence exercises a different behavior from the product tests. Critical transitions are tested at multiple layers: store-level invariants, HTTP behavior, real MCP client exchanges, mocked TrueForge sessions, and React DOM journeys. The UI tests exercise patient switching, help handoff, instruction review, source display, summary refresh, approval, response loss, retry, and emergency-link availability. A separate real-browser journey used a production build, real Express API, isolated SQLite store, and source-search mode.

The evaluation design names expected answers, required sources, allowed and prohibited tools, expected state changes, and failure categories. It covers missing laboratory details, conflicting uploaded records, cross-patient

requests, Synthea dosage overreach, attempted approval bypass, prompt injection embedded in a document, repeated reminder proposals, open help requests, tool failure, and runtime outage. Automated checks verify controls and state while human review remains responsible for answer quality.

Recorded integrated verification passed **59 backend and contract tests, four React DOM journeys, a production build, and 32 of 32 deterministic guardrail cases**. Real MCP SDK tests verified the summary tool and role-specific published inventories, including rejection of a reminder call through the summarizer endpoint. Mocked TrueForge tests covered model selection, role-specific specifications, structured output, existing-registration preservation, provider failure, invalid or foreign references, and a care plan changing during a run.

Provider-independent behavior is part of the product rather than an afterthought. If model execution is unavailable or rejected, Homeward makes no unauthorized tool calls or state changes, while deterministic next-step guidance, lexical source search, saved-record summaries, citations, and care-team workflows remain available. This fail-closed behavior keeps the useful parts of the recovery workflow operating without turning provider availability into authority over patient state.

Why the work is notable

Homeward's most impressive quality is the consistency of its safety model across interface, storage, tools, and evaluation. Many prototypes append citations to generated prose. Homeward instead makes source identity, review status, date provenance, plan version, and action authority part of the domain model. The distinction matters: a citation-shaped string is not accepted merely because a model emitted it; the backend must resolve it to the bound patient's current record.

The product combines provider-independent utility with bounded model assistance. Deterministic source search and summaries remain available at all times; a model can improve routing and selection when available. Human review governs operations that change the plan, while safe, read-only guidance remains responsive. Idempotent receipts make the visible failure demo a real reliability feature: the system demonstrates what happens when the network response is uncertain after a write, one of the operational edge cases that ordinary demos omit.

For patients and caregivers, the improvement is reduced fragmentation. The record, the next step, its source, open questions, and local reminder state live in one workflow. For a care-team operator, Homeward turns vague "I need help" signals into patient-scoped, source-linked queue items while preserving what was and was not resolved. For product and engineering teams, the capability matrix and evidence artifacts make it possible to discuss exactly which component authorized an action, which checks passed, and whether a result came from local records, a model, or a fallback. For healthcare organizations, the prototype offers a credible pattern for future integration: keep deterministic workflow authority around model assistance, preserve provenance, and measure live behavior separately from backend controls.

The evaluated environment uses fictional discharge material and synthetic Synthea history. Homeward continues development toward public launch; clinical outcomes and authenticated healthcare deployment require separate validation. Within that scope, the completed product is a detailed reference implementation for post-discharge coordination with source provenance, human approval, deterministic authority, and failure recovery built into the workflow.

Results and verified capabilities

Recorded results

1. The integrated verification passed 59 backend and contract tests, four React DOM journeys, a production build, and 32/32 deterministic guardrail cases.
1. The guardrail matrix covers missing details, conflicting records, cross-patient requests, dosage overreach, attempted approval bypass, prompt injection, repeated proposals, help pauses, tool failure, and runtime outage.
1. Provider rejection produced zero unauthorized tool calls or patient-state changes, demonstrating fail-closed behavior while deterministic record functions remained available.

Implemented capabilities

1. Source-linked recovery tasks, persistent completion/help state, deterministic next-step selection, accessible saved-record summaries, lexical source search, imported-document review, care-team review history, approval-controlled `.ics` creation, stale-action invalidation, and response-loss deduplication are implemented.
1. Four specialist roles use patient-and-role-scoped MCP inventories. Server enforcement rejects tools outside a role's inventory, and no agent has an approval tool.
1. Structured model references are validated against patient scope and current plan version; rejected output falls back to current records.
1. Synthea historical records, imported passages, authored discharge instructions, human clarifications, and general education remain distinct in the data and UI.
1. Runtime limits include one active run per patient, eight calls per minute, an eight-iteration AgentSpec limit, and a 90-second cancellation path.

What the results demonstrate

1. Homeward demonstrates a credible way to reduce the cognitive and coordination burden of turning discharge paperwork into follow-through while keeping uncertainty actionable and visible.
1. Its main technical contribution is an authority architecture: AI helps locate and organize evidence, while deterministic, auditable services retain control over instructions, dates, approvals, and side effects.
1. The product is strong evidence of product design, full-stack engineering, agent-tool contracts, reliability controls, and evaluation craft.

Source map

1. Product overview and implemented capabilities: `README.md`; architecture and runtime scripts in `package.json` and `.github/workflows/ci.yml`.
1. Core patient/care-team UI: `src/App.jsx`, `src/PatientSummary.jsx`, `src/ReviewWorkspace.jsx`, `src/styles.css`.
1. API and domain authority: `server/app.js`, `server/store.js`, `server/guidance.js`, `server/summary.js`, `server/agent-responses.js`.
1. Agent routing and TrueForge/MCP integration: `server/agents.js`, `server/trueforge.js`, `server/mcp.js`, `scripts/setup-trueforge.js`, `scripts/smoke-trueforge.js`.
1. Synthetic data, Synthea provenance, and education: `server/fixtures.js`, `data/synthea-sample.json`, `scripts/ingest-synthea.py`, `server/education.js`.
1. Design contracts: `docs/DISCHARGE-REVIEW-API.md`, `docs/NEXT-STEP-GUIDANCE.md`, `docs/AGENT-TEAM.md`, `docs/DATASET-AND-AGENT-INTEGRATION-GUIDE.md`, `docs/CARE-TEAM-UI.md`.
1. Test and evaluation methodology: `tests/`, `scripts/evaluate*.js`, `scripts/evaluations/`, `docs/LIVE-EVALUATIONS.md`, and `docs/VALIDATION.md`.
1. Recorded evidence: `docs/evidence/deterministic.json`, `docs/evidence/integrated-main/`, and `docs/assets/journey/`.

Birb Hangout - a shared place for distant friends to spend time together

What was built and why

Birb Hangout is a browser-based 3D social world built for friends and family who live far apart. Remote connection does not always need another feed, agenda, competitive loop, or text channel. Sometimes people want the digital equivalent of sharing a room - reading, choosing music, watching something, playing a game, resting by the fire, or setting off together. The product turns the idea into an inhabitable lounge where each person appears as a bird and the reward is the company itself.

That goal shaped the interface. Activities remain visible in the world instead of replacing it with full-screen menus. A bird browses the record cabinet, carries a sleeve to the turntable, and puts it on. A reader holds an open book while another bird remains visible across the room. The television, sofa, fireplace cushions, bookshelf, photo frame, wall switches, and trivia screen are spatial objects others can notice and approach. Focused overlays exist for setup - typing a video, selecting a title, answering a question - but the activity returns to the room. The result is not vaguely "multiplayer play." It is low-pressure co-presence: people share a setting while choosing different levels of attention and participation.

The implemented product has grown far beyond the repository's old scaffold description. It includes invite-code rooms; persistent room identity; customizable avatars; walking, seating, sleeping, emotes, and consent-based hugs; shared video and record playback; a room-owned vinyl cabinet and bookshelf; PDF and EPUB reading; voice chat; photo uploads; lighting and fireplace ambience; trivia with a synchronized physical ceremony; and a Dream reached from fireside rest. In the Dream, players fly through the same seeded landscape, follow one another, cross floating islands, and descend into rivers, clearings, and trees. Alongside the game, the repository contains a serious development environment: specialist studios, exact-time capture tools, spatial annotation, blind comparison packets, negative controls, candidate state machines, change ledgers, and visual-evidence gates.

Distant friends and family receive more ways to be together than either "talk continuously" or "play the same competitive game." Quiet and active moods coexist. One person can read while another explores; a third can join the sofa, ask for a hug, or settle by the fire. Shared objects make intention legible without forcing it through chat. The project also shows 3D teams how visual feedback can be grounded in a reproducible camera, object, world coordinate, and scene state rather than conveyed as an ambiguous sentence about "the thing near the left edge."

The end-to-end experience

A visitor creates a private room, chooses a name and bird appearance, and receives a short code embedded in the room URL. A friend opens that URL and joins the same Colyseus room. The code is more than a lobby label: it keys persistent room state, while a locally stored owner token allows the creator to reclaim host authority without requiring accounts. Fast refresh and reconnect use a stable browser client identity so a returning person reclaims the same live bird instead of appearing twice. A typed code that does not exist is rejected rather than silently creating an empty duplicate - an important response to the otherwise confusing failure mode where two friends believe they joined the same room but cannot see each other.

Inside, people navigate a furnished Three.js lounge and interact with spatial targets. Seating is authoritative and exclusive, so two players cannot occupy the same chair. The shared snapshot carries presence, position, pose, seat, avatar, room lighting, photo slides, vinyl records, shelf contents, media state, fireplace state, game state, and Dream membership. Local concerns such as camera mode and the private breathing interaction remain local. This distinction keeps the room coherent without synchronizing every transient UI choice.

The activities are implemented as visible sequences. Choosing music can begin at a shared cabinet, continue as a carried sleeve and wing animation, and resolve at the turntable. The cabinet belongs to the room rather than one browser's local storage. The sleeve can derive its thumbnail and an accent color sampled from image pixels, so the

physical prop reflects the selected media. Watching uses exactly one embedded video player shared by the TV and turntable paths. The server holds the selected video, status, shared playhead, duration, and a host synchronization token. One bird can temporarily hold the physical remote; everyone else follows the same state instead of issuing racing commands. Clients correct meaningful drift, reject stale player clocks, and re-anchor startup playback to a real reported playhead.

Reading solves two different rendering problems instead of compromising between them. The local reader needs crisp controllable pages; everyone else needs a believable bird, book, and pose in the world. A close reading view renders PDF pages and a deliberately bounded EPUB 2/3 subset onto textures. The EPUB path walks the container, package manifest, and spine, performs its own line wrapping and pagination, and maintains a ten-texture cache. Fixed-layout EPUBs are refused explicitly rather than rendered incorrectly. In the shared scene, a separate book actor stays attached to the bird's pose. The page-turn animation deforms a 25-by-17 vertex sheet, maps the following page correctly on the reverse, and avoids the common shortcut of rotating a rigid card or floating a second texture plane above the paper. Players can curate which of nine built-in books appear as physical shelf spines, and each reader advances privately while remaining visibly present to the room.

Social play spans quick reactions and coordinated events. The radial emote control is anchored to the local bird and supports hold, direction, and release rather than becoming another permanent panel. A hug is a separate interaction rather than just another emote: the initiator approaches a nearby bird, sees a short local offer, sends a private request, and the recipient accepts or rejects. The server checks identity, distance, availability, request rate, pair cooldown, and expiry; computes complementary target positions and facing; waits for both participants to report ready; and then starts one synchronized clip. Movement, cancellation, timeout, disconnection, and completion release both actors. The smoke specification uses three browser contexts to check that the offer and consent prompt stay private while the accepted clip is visible to the room.

Trivia makes a result into a shared piece of theatre. The server keeps the answer key off the live client schema, receives answers, scores the round, and broadcasts one ceremony start time. Every browser derives the current beat from elapsed time, allowing the TV reveal, cards, camera movement, and losing-bird ejection to remain aligned without a stream of choreography messages. Reduced motion is handled inside the same timeline. Voice uses WebRTC peer connections, with the server relaying signaling but never carrying audio; spatial gain makes distance audible without letting a faraway friend disappear entirely. The fireplace takes an equally crafted approach: crackles are synthesized by a recursively scheduled, randomized noise process rather than a loop with a seam, and the owner-tuned values were transferred from a browser sound lab into the game.

Rest is also an activity. A bird can nap on a fireside cushion, choose a sleeping posture, remain visible to passersby, and follow a private three-breath sequence into the Dream. The transition uses its waiting time: breathing animation, stars, camera treatment, and background loading progress together while the lounge body remains asleep. The server authorizes Dream entry only for an eligible napping player, owns one active Dream per room, mints the shared nonzero world seed, and tracks membership. Remote flight is sent as compact validated messages rather than inserted into the room snapshot, so awake lounge clients do not receive a high-rate position stream.

The Dream changes the scale without abandoning the product's social premise. Each browser advances its own bird through a fixed-step, dependency-light six-degree-of-freedom model. Pitch, roll, wing shape, and discrete flaps feed lift, induced and parasitic drag, gravity, momentum, and coordinated turning. Diving converts altitude into speed; pulling out redirects speed into climb; banking reduces vertical lift. Keyboard, touch, gamepad, and scripted probes resolve to the same control structure. Remote clients reconstruct a partner from motion packets with bounded extrapolation and correction against the same deterministic world.

The world itself is built from public-domain USGS 3DEP elevation data. A 900-metre terrain window is processed and extended into an approximately 3.8-by-8.8-kilometre range. Seeded builders create repeatable rivers, banks, floating landforms, shore, vegetation, and detail. A six-part cover budget divides each point among canopy, shrub, herb, grass, rock, and soil; the same normalized shares drive both ground color and planted geometry, preventing "forest-colored" ground with no forest. A committed species kit varies silhouettes through proportion, while level of detail, culling, tiling, and concentrated detail pockets keep the browser workload compatible with flight-scale terrain.

Architecture and implementation decisions

Birb Hangout is a TypeScript monorepo. The React 18/Vite client uses React Three Fiber, Three.js helpers, Rapier physics, and Zustand. The Colyseus server uses Express for health, ICE configuration, photo and room-management routes; Colyseus schemas for authoritative room state; and WebSocket messages for commands and transient streams. A shared package defines domain types, protocol payloads, constants, trivia content, and state defaults. The production Docker image builds all three workspaces and serves the compiled client from the same server origin, reducing deployment and CORS complexity.

The state model is intentionally mixed rather than universally synchronized. Durable facts use typed schemas and room records. High-rate Dream flight uses transient relays. Video playback uses authoritative intent plus local browser players. Voice media stays peer-to-peer. A ceremony uses one timestamp and deterministic local playback. A hug uses a server-owned request/staging/lifecycle protocol. This is a strong realtime design choice: each behavior uses the cheapest representation that still preserves the social invariant people notice.

Persistence supports local disk in development and S3-compatible object storage when configured. The same abstraction stores room records and private photo assets. Uploaded images require a short-lived bearer token issued only to a live room session, are limited by type and byte size, decoded under an input-pixel ceiling, autorotated, resized, re-encoded to WebP, and checked against a room quota. Room creation uses production-only per-device, per-network, and global budgets with keyed hashes rather than durable raw IP logs. WebSocket payloads are capped, message types are validated, and invalid-message counts can disconnect abusive clients. These controls do not make a production-security certification claim, but they show that the deployment was treated as a shared service rather than an unconstrained local demo.

Creative-engineering methodology

The most distinctive contribution may be the way the project learned to build and evaluate subjective 3D work. Visual quality is treated as an observable outcome with human authority, not something a passing typecheck can certify. The repository requires reference-led visual direction, before/after comparison, preserved rejected candidates, and owner sign-off in high-risk lanes. A single clearly defective frame can veto a candidate; a clean frame is not sufficient by itself. Tests are described by what they can establish - transport, timing, schema refusal, deterministic geometry - not promoted into claims about whether an animation feels alive or a landscape looks right.

Animation research produced concrete measurements before a broad retuning effort. Across the original twelve clips, every clip showed a roughly 1.3-to-1.5-world-unit boundary jump over about two frames; six contained dead holds, including several approximately 99 percent frozen clips; and nine lacked anticipation. The instrumented hug experiment preregistered eight likely failures and observed seven of them even though most had been predicted. The lesson was not that animation craft was mysterious. It was that guessed numbers, incomplete rig assumptions, and gates measuring the wrong construct were being allowed to survive.

The resulting candidate system turns iteration into a controlled comparison. A dev-only harness applies frame-keyed motion to an allowlisted set of rig controls, supports linear, cubic-Hermite, and hold interpolation, runs the same A/B/C structure for turntable clips and prop interactions, and restores the baseline atomically. Candidates must differ by mechanism, include an ablation and a reference-derived option, and report only metrics with a registered control. Exact-time captures, named action-axis cameras, orbit views, frame strips, telemetry sidecars, and manifests make runs reproducible.

The team then discovered that even a mechanically correct candidate packet could answer the wrong visual question. A book-pickup packet proved a bookshelf patch while the meaningful book remained merged with the wing. A switch animation showed a static handshake pose beside a self-moving switch. Those failures led to blind-first semantic review: an observer describes what action appears to happen before seeing the intended action; the contract names meaning-carrying cues, ordered events, causal links, counterfactual questions, and disconfirming conditions; only then is the observation compared with the intent. An applied transform is explicitly labeled transport evidence, not semantic completion. Owner uncertainty is handed to the owner instead of being converted into an agent rejection.

Specialist studios reduce each investigation to the smallest truthful environment. Birb Studio isolates the shipped avatar and emotes. Interaction Studio stages real actors and props. Room Studio exposes spatial dependencies. Rig

Studio runs the shipped bird, Dream placeholder, and candidate on one baked timeline so model comparisons do not drift out of phase. Dream and Landscape studios expose flight, camera, terrain, species, materials, and one-at-a-time landscape assets. These are not mockups: they import the game's components, pose functions, builders, and cameras behind development-only routes.

Spatial annotation closes the loop between human visual judgment and AI-assisted implementation. When the owner pauses a studio, orbits and zooms to the failing view, draws around the defect, and writes a comment, the overlay raycasts an 11-by-11 grid through the marked region. Geometry-derived subject IDs compete by vote; ambiguity and hit rate are retained rather than hidden. The durable note stores the frame hash, normalized drawing, camera pose and target, scene state, likely subject, alternate hits, world coordinates, subject-local coordinates, and the instruction needed to reopen the view. A later recapture uses the same camera and state. This converts "the cliff looks broken from this angle" into a bounded work item an AI collaborator can reproduce and compare.

The broader harness reinforces that discipline. Change ledgers preserve the owner request, affected lane, initial red tests, write set, and intentional omissions. Candidate safeguards freeze allowed and protected paths, references, metrics, roles, failure classes, and transition authority in a hash-chained state machine. Rejected mechanisms cannot quietly return under a new name; repeated failure in one class forces an owner decision. Visual review requires an independent unprimed observation, refuses byte-identical variant frames, carries open defects forward by the view where they were seen, and will not accept an after image without its before. The full research compendium documents these systems as evidence rather than retrospective mythology: character, flight, landscape, presence, activities, harness, and studios each separate implemented capability, measured result, human judgment, and held claim.

Results, significance, and beneficiaries

The strongest result is a coherent product across unusually different modes. The same identity can walk into a room, manipulate shared objects, read privately while remaining socially visible, negotiate media control, join a minigame, share a consensual hug, sleep, and enter a shared flight space. Server state, local presentation, and transient realtime streams remain distinct while still producing one legible experience.

The repository also records evidence of methodological improvement. Phase 2's structured animation infrastructure passed its recorded 11/11 initial self-test groups, 8/8 focused browser checks, 5/5 metric groups, 15/15 motion tests, and TypeScript checks before owner acceptance. After semantic-review remediation, the recorded candidate self-tests reached 34/34 groups and the focused browser suite 9/9. The owner rejected the first causal reading of the switch interaction, selected the corrected runtime-motion mechanism, and explicitly closed Phase 2. These are historical, commit-scoped results, not a claim that every current test passes.

The current source inventory contains 60 Playwright smoke-spec files and 199 top-level `test(...)` declarations spanning room lifecycle, media synchronization, reading, trivia, voice, hug privacy and synchronization, Dream sharing, flight motion, studios, HUD, mobile behavior, security boundaries, persistence, and visual workflows. That count describes the breadth of the implemented test surface rather than a single current pass result.

For players, the work reduces the pressure and fragmentation of remote socializing. For a family member who does not want a conventional game, the room offers recognizable rituals and quiet participation. For a more active player, the Dream offers skill, exploration, and shared landmarks. For designers, the project shows how diegetic objects can advertise activity and invite participation. For realtime engineers, it provides concrete patterns for matching state representation to behavior. For AI-assisted creative teams, its studios and annotation workflow reduce the expensive cycle of describing, misunderstanding, and rediscovering camera-dependent defects.

Author role and attribution

Neerav created and built Birb Hangout. He defined the product concept, social experience, interaction and world design, realtime architecture, and visual language; implemented and integrated the client, server, shared protocol, persistence, audio, media, reading, games, Dream, deployment, and safety controls; and designed the studios, spatial annotation, candidate comparisons, and evaluation methodology used to judge the work.

Evidence summary

Results with context

1. The original twelve animation clips were instrumented; the accepted research record reports 1.3-1.5 world-unit boundary jumps across all twelve, dead holds in six, and no anticipation in nine.
1. Seven of eight preregistered hug-experiment failure predictions were observed, motivating changes to the workflow rather than cosmetic retuning.
1. Recorded Phase 2 verification includes 11/11 candidate self-test groups, 8/8 focused browser checks, 5/5 metric groups, and 15/15 motion checks; the semantic remediation later records 34/34 candidate groups and 9/9 focused browser checks.
1. The current checkout contains 60 Playwright smoke-spec files and 199 top-level test declarations. This is an inventory, not a current pass count.

Implemented capabilities

1. Code-based room creation/join/rejoin, owner-token recovery, typed replicated room state, persisted records, shared objects, photo processing, media control, reading, trivia, voice signaling, consent-based hugs, naps, Dream membership, and shared-flight relays are implemented in source.
1. The lounge provides world-visible records, books, television, seating, fireplace, gestures, and game ceremonies instead of reducing these activities to disconnected full-screen interfaces.
1. The Dream uses a fixed-step flight model, shared seeded terrain, compact remote-flight messages, a USGS-derived heightfield, deterministic landscape builders, and a six-cover vegetation/ground model.
1. Development-only studios, exact-time capture, candidate comparison, visual review requests, change ledgers, hash-chained safeguards, and spatial notes with raycast-derived subject and coordinate data are implemented.
1. The repository has a single-service Docker/Render deployment design with optional S3 persistence and production-oriented rate, payload, image, and room limits.

What the evidence demonstrates

1. Birb Hangout demonstrates that a social world can optimize for co-presence rather than retention pressure, continuous conversation, or competition.
1. Its technical contribution is the integration of diegetic activity design, fit-for-purpose realtime synchronization, and reproducible human-in-the-loop 3D evaluation.
1. The studios and spatial annotation system are a meaningful automation improvement: they turn subjective, viewpoint-dependent feedback into a durable, reopenable, comparable engineering artifact.
1. The project is strong evidence of product thinking, realtime full-stack engineering, interaction design, procedural 3D work, and evaluation design. It does not by itself establish large-scale retention, population-level social benefit, high-concurrency capacity, or universal visual acceptance.

Exact source map

1. Product framing and activity journeys: live About site, `https://birb-hangout-about.onrender.com/`; `docs/birb-lounge-product-spec.md`; `docs/thesis/SITE-COPY-v7.md`.
1. Repository map, risk lanes, current and historical evidence pointers: `README.md`, `docs/INDEX.md`, `AGENTS.md`, `docs/workflow-protocol.md`.
1. Client composition and entry flow: `client/src/App.tsx`, `client/src/ui/RoomEntry.tsx`, `client/src/game/scene/LoungeScene.tsx`, `client/src/ui/Hud.tsx`, `client/src/lib/useRoomStore.ts`.

1. Shared state and protocol: `shared/src/types.ts`, `shared/src/protocol.ts`, `shared/src/constants.ts`, `server/src/schema/`, `server/src/rooms/BirbLoungeRoom.ts`, `server/src/rooms/triviaController.ts`.
1. Persistence, upload processing, traffic bounds, deployment: `server/src/persistence/roomPersistence.ts`, `server/src/security/`, `server/src/index.ts`, `server/src/config/env.ts`, `Dockerfile`, `render.yaml`, `.env.example`, `docs/DEPLOYMENT.md`.
1. Media, audio, reading, and games: `client/src/game/media/youtubeController.ts`, `client/src/game/media/vinylLibrary.ts`, `client/src/game/audio/`, `client/src/game/scene/ReadingBookView.tsx`, `client/src/game/scene/readingDocument.ts`, `client/src/game/scene/readingBookContent.ts`, `client/src/game/scene/triviaTheme.ts`, `client/src/game/trivia/ceremonyTimeline.ts`.
1. Hugs and expression: `client/src/game/scene/BirbSocialInteraction.tsx`, `client/src/game/scene/BirbAvatar.tsx`, `client/src/ui/EmoteDial.tsx`, `shared/src/protocol.ts`, `server/src/rooms/BirbLoungeRoom.ts`, `tests/smoke/hug-interaction.spec.ts`.
1. Dream and flight: `client/src/game/dream/DreamPortalHost.tsx`, `client/src/game/dream/dreamEntryTimeline.ts`, `client/src/dream/flightModel.ts`, `client/src/dream/flightParams.ts`, `client/src/dream/flightInput.ts`, `client/src/dream/peerReplication.ts`, `server/src/schema/DreamStateSchema.ts`, `docs/dream-shared-plan.md`, `docs/dream-flight-physics-findings.md`.
1. Landscape: `client/src/dream/demWorld.ts`, `client/src/dream/rangeCover.ts`, `client/src/dream/landscape/`, `tools/dem/`, `client/public/dream/zion-190.json`, `docs/reference-images/dream-landscape/CANON.md`, `docs/probes/dream-landscape-hash.mts`.
1. Animation findings and accepted Phase 2 record: `docs/animation-authoring-research-2026-07-26.md`, `docs/animation-hug-experiment-log-2026-07-27.md`, `docs/animation-phase-2-evidence-2026-07-28.md`, `docs/animation-phase-2-semantic-review-evidence-2026-07-29.md`, `client/src/game/candidateHarness.ts`, `tools/candidate-round.js`.
1. Studios, capture, and spatial annotation: `docs/studios.md`, `docs/studio-notes.md`, `docs/visual-reference-workflow.md`, `client/src/studio/notes/`, `client/src/studio/RigStudio.tsx`, `client/src/dream/DreamStudio.tsx`, `client/src/dream/LandscapeStudio.tsx`, `tools/studio-capture.js`, `tools/anim-montage.js`, `tools/visual-evidence-gate.js`.
1. Research synthesis: `docs/thesis/BIRB-HANGOUT-RESEARCH-COMPENDIUM.pdf` (301 pages), especially D2 Character, D3 Flight, D4 Landscape, D6 Presence, D7 Activities, D8 Harness, and D9 Studios, with corresponding source packets under `docs/thesis/handoffs/` and evidence records under `docs/thesis/claims/`.

Four-class Alzheimer MRI classification: CNN experimentation and probability ensembles

Study and evidence context

This record describes the public repository `neerav-git/Alzheimer-s-Disease-Detection-Ensemble-Learning`, inspected on October 12, 2026 at `main` commit `2bae1dd`. The repository contains one 100-cell Google Colab notebook, `AZclassification5(wide).ipynb`, and a README that explains how to supply the external image data and rerun the workflow. The notebook was created in April 2024; the README was expanded in May 2026. The full public commit history contains six commits, all attributed to Neerav identities.

Retained notebook outputs establish the data count, class names, batch partitions, model summaries, and visible MRI examples. The executable cells document the preprocessing, modeling, inference, and ensemble procedures. Performance statements retain their named provenance so the public notebook result and companion experiment remain distinct.

This is a separate project from *Analyzing Brain Atrophy in Dementia Using Computer Vision*, the five-page IEEE ICRITO 2022 paper by Neerav Chaudhary and Sourabh Singh Verma. The paper studies longitudinal atrophy measurement with image processing; this repository studies four-class single-image classification with TensorFlow CNNs and prediction-level ensembles. Neither project's evidence should be used to fill gaps in the other.

The problem: make multi-stage MRI classification experimentation inspectable

The practical machine-learning problem is to distinguish four folder-defined stages from brain MRI images: `NonDemented`, `VeryMildDemented`, `MildDemented`, and `ModerateDemented`. This is more demanding than a binary normal-versus-dementia demonstration because the model must learn boundaries between adjacent severity categories whose images can appear structurally similar. It also creates an evaluation question beyond "can a CNN fit these images?": can independently saved models be combined so that one model's uncertain or idiosyncratic prediction is moderated by the others?

The repository addresses that question as an end-to-end experimental notebook rather than as an isolated architecture snippet. It begins with data access and visual inspection, turns a directory tree into TensorFlow batches, partitions those batches, adds training-only augmentation, defines and inspects a CNN, supports single-image and grid inference, reloads independently saved models, and compares single-model evaluation with two-model, three-model, and draft weighted ensembles. The contribution is the experimental progression. A reader can see how the input data becomes tensors, how the tensors become class probabilities, and how those probabilities become an ensemble decision.

This work is useful first to applied ML researchers and students who need a concrete, modifiable four-class imaging workflow. It is also useful to research teams deciding whether model diversity merits further controlled study: the notebook provides the full scaffolding needed to compare variants on a common task. Patients and clinicians are the motivating population, but they are not users of a deployed system here. The work is a retrospective classification experiment, not a clinical diagnostic service.

What was built: a complete notebook workflow over 6,400 images

The saved directory-loader output reports exactly **6,400 files belonging to four classes**. With a batch size of 64, the dataset contains 100 batches. The class-name output records the alphabetical folder order as `MildDemented`, `ModerateDemented`, `NonDemented`, and `VeryMildDemented`. A retained batch inspection confirms tensors shaped `(64, 256, 256, 3)` and integer class labels. The notebook then displays a 12-image labeled montage so the investigator can inspect actual inputs before training rather than treating the folder tree as an abstract table.

There is also a small but thoughtful image-content check. Two OpenCV/NumPy approaches measure black background pixels in sampled scans. Saved outputs show a black-area fraction of `0.480087...` for one `VeryMildDemented` sample and `46.47%` for one `NonDemented` sample. These are not dataset-wide quality statistics, but they demonstrate attention to a characteristic of MRI slices that can dominate pixels while carrying no anatomy. This kind of exploratory inspection is important when choosing resizing, convolution, and augmentation behavior.

The declared partition is 80% training, 10% validation, and 10% test. The retained outputs show 80, 10, and 10 batches respectively - corresponding to 5,120 training images, 640 validation images, and 640 test images for this 6,400-file, full-batch run. The pipeline caches each partition, shuffles it, and prefetches with `tf.data.AUTOTUNE`, so data preparation can overlap model execution instead of becoming a serial input bottleneck.

Images are resized to 256x256 and rescaled by `1/255`. Training data is augmented with horizontal and vertical flips, rotation with factor `0.2`, width and height variation with factor `0.2`, and zoom with factor `0.1`. The augmentation is mapped onto `train_ds`; validation and test datasets are not sent through that augmentation map. This automates the generation of varied training views without manually creating and storing duplicate images.

The main CNN recorded in the current notebook contains six convolutional blocks with 64, 64, 128, 256, 256, and 256 filters. Every convolution is followed by 2x2 max pooling. The classifier head uses 30% dropout, flattening, a 512-unit ReLU dense layer, and a four-unit softmax output. The saved Keras summary reports **2,114,756 trainable parameters**. The intended training configuration is Adam optimization, sparse categorical cross-entropy, and accuracy tracking. The notebook includes history plots for training/validation accuracy and loss, a sample prediction cell, and a reusable inference function that returns both the predicted class and maximum softmax confidence for a 3x3 review grid.

The workflow also preserves an architectural contrast. A previously saved model loaded from the `model3.0` Drive directory has six convolutional layers with 32, 64, 64, 128, 128, and 128 filters, followed by dropout and a 128-unit dense layer. Its retained summary reports **491,524 trainable parameters**, less than one quarter of the 2.11-million-parameter model. Two other saved models are loaded as `modelv1` and `modelv2`. Although their summaries are not preserved in the public file, the use of separately saved variants makes the ensemble experiment materially different from duplicating one prediction three times.

The notebook's error-review method is qualitative and image-level. For both a single saved model and the three-model ensemble, it defines 3x3 grids whose titles place the actual folder label, predicted label, and maximum softmax confidence directly above each scan. This gives the investigator a fast way to inspect *what kind* of image produced a correct or incorrect decision and whether an error was made with high or low confidence. Those grid outputs are not retained in the public notebook, and the code does not aggregate them into a confusion matrix or class-wise rates. It is therefore accurate to describe implemented sample-level prediction review, not a completed statistical error analysis.

The ensemble contribution: combine model judgments at the probability level

The strongest implemented research idea is late fusion of class probabilities. The two-model ensemble feeds a common 256x256x3 input through `new_model` and `modelv2`, then applies Keras `Average` to the two four-class output vectors. The three-model ensemble repeats the procedure with `new_model`, `modelv1`, and `modelv2`. Because softmax vectors encode each model's relative support for all four classes, averaging uses more information than majority voting over final labels. A confident wrong prediction can be counterbalanced by two models assigning probability mass elsewhere, while agreement is reinforced.

This approach improves the experimental workflow as well as the model result. Without it, comparing trained variants is often a manual sequence of loading a checkpoint, running evaluation, writing down a score, and mentally deciding which model to keep. Here, saved models become composable Keras graph components evaluated through the same test pipeline. The ensemble is itself compiled and evaluated with the same sparse cross-entropy and accuracy metric, and it can be passed into the same visual inference function as a single model. That makes model combination a reproducible operation within the notebook rather than an after-the-fact claim.

The final section explores weighting rather than equal averaging. It defines a custom Keras layer with proposed weights `0.3`, `0.6`, and `0.1`. The current `call` implementation combines only the first two inputs and does not apply the third weight, so the defensible description is a **draft weighted-ensemble experiment**, not a completed three-model weighted result. Its presence still shows the next methodological question the work was testing: whether the stronger constituent should have more influence than its peers.

Methodology and evaluation provenance

The notebook follows a sensible iterative methodology: inspect the inputs; standardize their dimensions and numeric range; introduce controlled variation only in the training stream; establish a convolutional baseline; preserve trained variants; then compare output fusion strategies on a held-out partition. Keeping the model summaries, sample images, architecture rendering, inference views, and evaluation calls in one document makes the reasoning inspectable. The Git history reinforces that this was an active experimental record: an initial notebook commit was followed minutes later by corrections to an indentation failure and expanded dataset framing, rather than a polished result being uploaded without its working context.

The result record has two distinct sources that should remain distinct:

1. In the public notebook, immediately after `ensemble_model3.evaluate(test_ds)`, a markdown cell says: **"A jump in accuracy of 13% has been observed when comparing the ensembled model with the original."** The cell does not say whether 13% means percentage points or relative percent. The corresponding `model.evaluate` output is not saved, so the public repository preserves the result statement and matching evaluation code, but not the numeric score pair or run log behind it.
1. The portfolio source ledger records an **author-confirmed companion experiment result of 78% baseline accuracy and 85% ensemble accuracy**. That is a seven-percentage-point increase, or about 9% relative to the 78% baseline. The current public repository does not contain or link the companion notebook's output cells, so 78%->85% should be attributed to the companion record rather than described as reproduced from the public notebook.

These statements may refer to different baseline/ensemble comparisons or different ways of calculating improvement; the available evidence does not justify treating them as interchangeable. The positive, exact account is therefore: the public artifact implements and evaluates equal-averaging ensembles and records a 13% improvement statement; a separate author-confirmed companion experiment reports a 78% to 85% held-out accuracy improvement. Neither result was rerun during this documentation pass.

The retained public outputs independently verify several important experimental facts: 6,400 files, four classes, 100 total batches, an 80/10/10 batch partition, 256x256 RGB-shaped tensors, a 2,114,756-parameter main CNN, and a 491,524-parameter saved CNN. The notebook also directly records visual data inspection and architecture rendering. What it does not preserve is a confusion matrix, per-class precision/recall, repeated-seed statistics, or the numeric outputs of its `evaluate` cells. Accordingly, the result is evidence of a successful research iteration, not a claim of validated diagnostic performance.

One evaluation detail matters for future interpretation. Partitioning is performed over a shuffled TensorFlow dataset of images/batches, and the repository does not include patient identifiers or a subject-level split manifest. Multiple slices from one person could therefore be split across partitions if the source folder contains them; the public code cannot establish otherwise. This does not erase the experiment's value, but it defines what the accuracy numbers measure: performance on the notebook's held-out image pipeline, not proven generalization to unseen patients, scanners, or clinical settings.

Why the work is notable

Within a compact Colab artifact, the author assembled the major stages of a real computer-vision study instead of stopping at a tutorial classifier. The notebook handles external storage, GPU-aware TensorFlow setup, directory-based labels, visual quality inspection, performant input pipelines, augmentation, a deep CNN, checkpoint loading, architecture visualization, reusable inference, and prediction-level fusion. It compares a 2.11M-parameter network with a saved 491K-parameter design and treats model variation as a resource for ensemble construction. That breadth is especially valuable in notebook-first research, where the hardest work is often connecting many individually familiar operations into an experiment another person can follow.

The ensemble design is also appropriately simple for the evidence available. Rather than introducing a complex meta-learner that would require another training protocol, equal averaging uses existing model probabilities and can be evaluated immediately on the same held-out stream. It creates a clear experimental contrast: single model versus two-model consensus versus three-model consensus. The reported improvements suggest that the constituent models captured complementary decision boundaries, which is the core reason ensembling can outperform an individual network. That complementarity is an interpretation of the observed comparison, not a separately measured diversity statistic.

For an applied research team, the workflow reduces repeated image handling and comparison work. TensorFlow automatically discovers the files, batches and prefetches them; the augmentation layer generates variations during training; the inference helper converts raw predictions into class/confidence labels; and the Keras functional API fuses checkpoints into a single evaluable graph. For a reviewer, the notebook makes the sequence legible through retained examples, summaries, and code. For a future medical-imaging study, it supplies a practical starting point for subject-level splitting, class-wise error analysis, external validation, and calibration - without claiming those later steps are already complete.

Author role and attribution

The public history supports describing Neerav as the sole repository author and builder of the recorded experiment. Four April 2024 commits under the `Neerav21` identity created and corrected the Colab notebook. Two May 2026 commits under `neerav-git/Neerav` expanded and then tightened the README. There are no other commit authors in the repository.

The defensible role statement is: Neerav designed and implemented the notebook's data-loading, image-inspection, preprocessing, augmentation, CNN, inference, checkpoint-loading, and ensemble-comparison workflow, and later documented how to run it. The repository does not establish authorship of the underlying MRI dataset or clinical labels, and it does not identify a clinical collaborator. The separate IEEE atrophy paper has a second human author, Sourabh Singh Verma, but that team attribution belongs to the publication, not to this four-class repository.

Evidence summary

Results with context

1. The committed loader output reports 6,400 image files in four classes; the dataset produces 100 batches of 64 at 256x256x3.
1. Retained partition outputs show 80 training, 10 validation, and 10 test batches, corresponding to 5,120/640/640 images in this run.
1. The main CNN summary reports 2,114,756 trainable parameters; the loaded `model3.0` summary reports 491,524.
1. The public notebook records a "13%" accuracy jump for the three-model ensemble versus the original, without saved score outputs or a stated percentage-point/relative-percent convention.
1. The author-confirmed companion experiment record reports 78% baseline accuracy and 85% ensemble accuracy on its held-out split: +7 percentage points, approximately +9% relative. This result is not reproduced in the public notebook's retained outputs.

Implemented capabilities

1. Directory ingestion, four-class label discovery, MRI montage inspection, 256x256 resizing, `1/255` rescaling, and training-only augmentation are implemented.
1. An 80/10/10 dataset-partition function, cache/shuffle/prefetch pipeline, six-block CNN, softmax inference, confidence display, and architecture visualization are implemented.
1. The notebook loads three saved Keras models, constructs equal-average two- and three-model ensembles, compiles them, and calls evaluation on the test dataset.
1. A weighted-ensemble layer is prototyped; its current forward pass uses two of the three model outputs, so it is not evidence of a completed three-model weighted ensemble.

What the evidence demonstrates

1. The work's strongest contribution is the connected experimental progression from raw folder data to reusable probability ensembles, not a novel convolution operator.
1. The reported ensemble gains are consistent with separately trained CNN variants contributing complementary errors, although diversity was not measured directly.
1. The workflow automates data preparation, variation generation, checkpoint reuse, prediction display, and probability fusion, reducing manual experiment assembly.
1. The study demonstrates a promising retrospective classification method. It does not establish unseen-patient generalization, external-dataset performance, or clinical diagnostic readiness.

Source map

1. Current public repository and run instructions: `README.md`, inspected at commit `2bae1dd0b56d92acdff9f3e0969f680fe5b8c8a5`.
1. Primary experiment artifact: `AZclassification5(wide).ipynb`, especially cells 12-18 (configuration, loader, classes, batch shape), 20-24 (image and background inspection), 26-41 (partitioning, caching, resizing, augmentation), 44-70 (CNN, summaries, evaluation/inference, architecture view), and 72-99 (model persistence, saved variants, equal and weighted ensembles).
1. Exact retained notebook outputs: cell 16 (6,400 files/four classes); cell 17 (class order); cell 18 (64x256x256x3 batch); cells 26-33 (100 total and 80/10/10 partition batches); cell 47 (2,114,756 parameters); cell 76 (491,524 parameters).
1. Public result statement: notebook cell 93, following the three-model evaluation call in cell 92, records the 13% improvement wording. Evaluation cells 52, 77, 83, 87, 92, and 99 have no saved output in the current public file.
1. Companion-result provenance: `docs/CONTENT_SOURCE_LEDGER.md`, "Four-class Alzheimer MRI classification," records the author-confirmed 78% baseline and 85% ensemble result and the public-output boundary.
1. Repository history: `7795f1e` (initial README), `f1556e9` (initial notebook), `9f70dc0` (remove retained indentation failure/correct compile cell), `32e5471` (expand four-class dataset framing), `d194441` (detailed README), and `2bae1dd` (tighten README).
1. Separate publication, not evidence for this classification experiment: Neerav Chaudhary and Sourabh Singh Verma, *Analyzing Brain Atrophy in Dementia Using Computer Vision*, IEEE ICRITO 2022, DOI `10.1109/ICRITO56286.2022.9964994`.

Analyzing Brain Atrophy in Dementia Using Computer Vision

Publication and study context

This is a full-paper work record based on a complete reading and visual inspection of all five pages of the IEEE conference paper, including its two tables, seven figures, four equations, and fourteen references. It is evidence for a later portfolio copy pass, not public-site copy.

Neerav Chaudhary and Sourabh Singh Verma authored "Analyzing Brain Atrophy in Dementia Using Computer Vision" while affiliated with the Department of Computer and Communication Engineering at Manipal University Jaipur. The paper was presented in the 2022 10th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions), held at Amity University in Noida, India, on October 13-14, 2022. IEEE added the record to Xplore on December 8, 2022. Its DOI is 10.1109/ICRITO56286.2022.9964994. The downloaded paper is five pages long and numbers those pages 1-5 internally; the conference table of contents places it at proceedings pages 1234-1238.

The work is best described as an interpretable medical-image analysis experiment. It converts intensity information from T1-weighted brain MRI images into a simple quantitative score, called `dem`, that is intended to represent visible tissue loss relative to dark regions associated with cerebrospinal fluid, sulcal widening, and ventricular expansion. The study then derives lower and upper reference bounds from labeled image groups, tests the score as a dementia classification signal, and compares a whole-brain/default crop with a hippocampal crop. The reported result is a five-percentage-point accuracy improvement, from 66% with the default crop to 71% with the hippocampal crop.

The human and research problem

Dementia is not a single visual event. It is a progressive condition whose effects can appear as structural changes in the brain alongside declining memory, motor function, language, reasoning, and other cognitive abilities. The paper focuses on a concrete imaging problem inside that larger clinical context: researchers need a way to track and quantify atrophy, but established manual tracing and segmentation workflows require expert labor, take substantial time, and can be difficult to reproduce consistently across large image sets.

The paper identifies several structural patterns associated with dementia research: thinning of the cerebral cortex, reductions in subcortical volume, widening of the sulci, shrinkage of the gyri, ventricular expansion, hippocampal tissue loss, and white-matter lesions. Some structural change is expected in normal aging, so the research challenge is not merely to find any atrophy. It is to create consistent measurements that can help compare cognitively normal and cognitively impaired images and, eventually, measure change over time.

That framing matters because it turns a broad aspiration such as "use AI for dementia" into an operational question: can a repeatable computer-vision pipeline extract a transparent, image-level measurement from MRI scans without requiring an expert to trace every region by hand? The study's target is the expensive measurement step. It is not presented as a complete clinical diagnostic system.

The implemented research contribution

The main contribution is a Python/OpenCV-style image-processing method that reduces each MRI image to interpretable pixel-density measurements and combines them into a ratio. Instead of training an opaque end-to-end neural network, the method uses explicit color thresholds and an explicit equation. A reader can follow the transformation from image to binary masks, from masks to pixel percentages, from percentages to the `dem` score, and from scores to classification regions.

The system uses three intensity categories:

1. black, represented as `[0, 0, 0]` and denoted `b`;
1. white, represented as `[255, 255, 255]` and denoted `w`;
1. grey, represented by a channel-wise interval bounded by `[70, 70, 70]` and `[190, 190, 190]`, denoted `g`.

For each color interval, the algorithm creates a binary mask. Pixels within the interval are marked as 1 and pixels outside it as 0. Counting the selected pixels and dividing by the relevant image area yields the percentage assigned to each category. The first measurement equation is:

```
dem = b / (w + g)
```

The numerator treats dark area as a proxy for the cerebrospinal-fluid and open-space patterns that increase with ventricular expansion and sulcal widening. The denominator treats white and grey pixels as a proxy for retained brain tissue. A higher `dem` value therefore means more dark area relative to visible tissue and, under the paper's model, a greater likelihood or extent of atrophy associated with dementia.

The score is deliberately simple. That simplicity is useful: every output can be traced back to pixel counts and a published equation. It also makes image preparation and cropping consequential. Black background outside the brain can influence the numerator unless the image is consistently framed; focusing the analysis on a clinically relevant area can reduce irrelevant pixels and make the signal more specific. The study's whole-brain versus hippocampal-crop result is evidence that region selection materially affects this handcrafted measure.

Dataset and study design

The study uses T1-weighted MRI images from the Alzheimer's Disease Neuroimaging Initiative (ADNI). In T1-weighted scans, the paper describes outer grey matter as darker than inner white matter, while cerebrospinal fluid appears dark. Those contrast properties are what make the color-mask approach possible.

Table I reports 2,198 images divided into two analysis groups:

1. 1,172 images with Clinical Dementia Rating (CDR) score 0, labeled cognitively normal or non-demented;
1. 1,026 images with CDR scores 0.5-2, grouped as cognitive impairment or dementia.

The unit reported by the paper is images, not unique people. The paper does not state how many subjects produced those images, whether multiple slices or visits from the same subject appear in both development and evaluation data, or how the images were split for model development and accuracy calculation. Therefore, 2,198 is a verified image count, not a verified patient count.

Figures 1-3 visually establish the spectrum the method is meant to analyze: an axial MRI labeled cognitively normal, an image labeled mild cognitive impairment, and an image labeled Alzheimer's disease/dementia. Figure 4 adds a schematic comparison showing wider sulci, smaller gyri, and enlarged ventricles in a demented brain. The figures do more than decorate the paper: they connect the pixel-ratio design to a specific anatomical story. More dark space and less visible tissue should increase `b / (w + g)`.

The study combines two related experiments. First, it evaluates the `dem` score as an image-level signal for separating labeled groups. Second, it tests whether narrowing the analysis to a hippocampal crop improves classification accuracy compared with the default crop. It also defines a longitudinal atrophy-rate equation for multiple scans of the same subject, but it does not report a paired longitudinal cohort or a measured longitudinal result in the Results section. The longitudinal calculation is therefore an implemented methodological extension described by the paper, while the published accuracy results are image-classification results.

End-to-end workflow

The research workflow can be reconstructed as seven stages.

1. **Select and label MRI images.** T1-weighted ADNI images are organized using their diagnostic/CDR grouping: CDR 0 for cognitively normal and CDR 0.5-2 for cognitive impairment/dementia.

1. **Choose the analysis region.** The study evaluates a default crop and a crop around the hippocampus. This is a meaningful design decision because hippocampal shrinkage is associated with memory impairment and dementia progression, and a tighter crop changes the balance between anatomical pixels and irrelevant background.
1. **Generate binary masks.** OpenCV color detection is applied separately for black, white, and grey intensity ranges. The masks transform continuous image appearance into countable pixel sets.
1. **Calculate pixel densities.** The pipeline counts pixels selected by each mask and expresses them as percentages `b`, `w`, and `g`.
1. **Calculate the image-level atrophy proxy.** The pipeline computes `dem = b / (w + g)`. The output is a continuous score rather than only a class label. Figure 6 shows a mildly demented scan with a calculated `dem` value of 0.60.
1. **Derive group bounds and classify.** The mean `dem` for the labeled dementia images becomes the upper reference bound:

```
dem_high = sum(dem for demented images) / total demented images
```

The mean `dem` for the non-demented images becomes the lower reference bound:

```
dem_low = sum(dem for non-demented images) / total non-demented images
```

The methodology defines three regions: below `dem_low` is non-demented; at or above `dem_high` is dementia; and values between the two bounds are intermediate or undetermined, with likelihood interpreted by proximity to the bounds. This third region is notable because the system does not force every image into a confident binary answer.

1. **Optionally calculate change over time.** For repeated scans of the same subject, the paper defines:

```
atrophy rate = (dem_final - dem_initial) / time interval
```

This final equation turns the image-level score into a longitudinal rate. It supplies a direct route from consistent scan processing to temporal measurement, although the paper's Results section does not present longitudinal rate statistics.

Figure 5 summarizes the computational pipeline as image input, binary-mask generation for black/white/grey ranges, pixel-density extraction, and atrophy-rate calculation. It is a compact, reproducible architecture with few moving parts: MRI image, region crop, threshold masks, counts, ratio, cohort bounds, and optional time normalization.

Results and strongest evidence

The results section reports that the model correctly classified some images as demented or non-demented and left others undetermined. That behavior is consistent with the three-region decision rule rather than a forced binary classifier. The authors observed a correlation between higher `dem` values and dementia labels.

Three quantitative results anchor the paper:

1. A worked mildly demented MRI example produced `dem = 0.60`.
1. The group-derived lower bound `dem_low` was 0.41, and the upper bound `dem_high` was 0.83.
1. Accuracy was 66% with the default crop and 71% with the hippocampal crop, a five-percentage-point absolute improvement.

Figure 7 plots image-level `dem` values against the two horizontal bounds. The plot makes the decision logic inspectable: low scores fall below the green lower line, high scores exceed the brown upper line, and intermediate cases occupy the area between them. The Results prose contains one imprecise sentence saying images "above the lower bound" are classified as demented; the formal methodology directly above it says dementia begins at or above the upper bound, with the region between bounds treated as varying likelihood. The formal three-region definition is the coherent rule and should be used when describing the system.

Table II reports the 66% and 71% crop comparison. Its leftmost column is titled "CDR Score" and contains 1 and 2, while the accompanying prose describes the rows as a comparison between default and hippocampal crops. The

defensible result is the crop/accuracy pairing. The table does not provide enough information to claim class-specific accuracy by CDR stage.

The paper also concludes that the process is time-efficient relative to manual tracing, but it does not report elapsed time, expert-hours saved, or a controlled timing comparison. A strong portfolio statement can say that the pipeline automates pixel measurement and avoids manual tracing inside this analysis; it should not attach a percentage or number of hours to the efficiency gain.

Why the work is technically notable

The project is notable first for turning an anatomical hypothesis into an auditable computation. The research narrative is not "a model found a pattern." It specifies which visual changes matter, how those changes map to intensity categories, how the categories become a numerical feature, and how the feature becomes a decision region. That chain makes the system easier to explain, debug, and challenge than an end-to-end prediction with no explicit measurement model.

Second, the work tests spatial focus rather than assuming that more image area is always better. The hippocampal crop improved reported accuracy from 66% to 71%. This supports the practical insight that clinically motivated region selection can improve a simple vision pipeline by reducing irrelevant image content and concentrating measurement on anatomy associated with memory and dementia. The experiment also exposes an important engineering truth: preprocessing is part of the model. A different crop changes the pixel distribution, which changes `dem`, which changes classification.

Third, the system provides a continuous, reusable intermediate representation. The `dem` score can be plotted, averaged by group, bounded into classes, or compared across time. That is more useful for research iteration than a single terminal label because it lets an investigator inspect how far an image lies from group references and see how a processing change affects the measurement.

Fourth, the pipeline is computationally modest. Binary masks, pixel counts, ratios, and cropping can run without a large learned model. That makes the approach suitable as a transparent baseline, a teaching tool, or a quick screening experiment before investing in heavier segmentation and deep-learning systems. The paper does not establish deployment performance, but it does demonstrate a complete research loop from clinical observation to engineered feature to empirical comparison.

Who benefits and what effort it reduces

The immediate beneficiaries are medical-imaging researchers and students working with large collections of MRI images. The pipeline converts a visual review task into repeatable operations, allowing the same thresholds and equations to be applied across images rather than asking an expert to redraw every boundary. It can accelerate hypothesis testing about which regions, crops, or intensity rules carry useful signal.

Researchers studying longitudinal imaging could also benefit from the defined rate equation. Once scans are consistently aligned, cropped, and processed, the same score can be compared across time points. The contribution is the measurement mechanism and its temporal extension, not proof that the resulting rate is a validated clinical biomarker.

Clinical and radiology teams are potential downstream beneficiaries only after substantially stronger validation. A transparent quantitative feature might eventually support review by highlighting change or prioritizing cases, but the paper does not establish safety, sensitivity, specificity, external generalization, or patient-care outcomes. The current work should be framed as retrospective imaging research that reduces analysis friction, not as an autonomous diagnosis or treatment recommendation.

Software and ML practitioners also benefit from the methodology as an example of interpretable feature engineering. It shows how domain knowledge can guide region selection and how an uncertain middle region can be preserved instead of forcing confidence. These ideas transfer to other image-analysis tasks even when the exact thresholds and `dem` equation do not.

Authorship and role attribution

Neerav Chaudhary is the paper's first author; Dr. Sourabh Singh Verma is the second author. Both are credited by IEEE and listed with the same Manipal University Jaipur department affiliation. The publication itself does not include a contributor-role taxonomy, so it supports co-authorship but not a detailed division of writing, study design, coding, or supervision.

Neerav's portfolio source ledger and resume provide narrower role corroboration: he built a repeatable Python/OpenCV pipeline for MRI preprocessing and brain-atrophy quantification, converting scans into consistent structural measurements and supporting the published IEEE study. That is a strong, concrete individual contribution. The paper's overall research design, interpretation, and publication outcome should continue to be attributed to both authors unless additional contributor records establish a more specific split.

Research lineage

The fourteen references show that the work was situated at the intersection of dementia research, neuroanatomy, segmentation practice, and image-processing implementation. References 1-7 establish neurochemical and structural change, the value of early intervention, treatment context, cortical/subcortical atrophy, white-matter dementia, and language/cognitive impact. References 8-13 ground the hippocampus, manual hippocampal segmentation, semi- and fully automated segmentation, reproducibility of atrophy-rate algorithms, ADNI clinical characterization, and sulcal-span changes. Reference 14 supports the use of Python/OpenCV color detection.

This reference structure is important to the work's credibility. The authors did not choose a color ratio arbitrarily: the ratio is motivated by visible structural features described in the literature, and the hippocampal crop is connected to an anatomical region already central to dementia research.

Evidence summary

Results with context

1 The analyzed dataset contained 2,198 ADNI T1-weighted MRI images: 1,172 CDR 0 images and 1,026 CDR 0.5-2 images.

1 The system generated black, white, and grey binary masks, calculated pixel percentages, and computed `dem = b / (w + g)`.

1 The reported lower and upper group bounds were 0.41 and 0.83.

1 A displayed mildly demented scan produced a `dem` value of 0.60.

1 The reported accuracy was 66% for the default crop and 71% for the hippocampal crop.

1 The paper is a five-page, two-author IEEE ICRITO 2022 conference publication with DOI `10.1109/ICRITO56286.2022.9964994`.

Implemented capability and contribution

1 The workflow automates mask generation, pixel-density measurement, score calculation, and bound-based classification for MRI images.

1 The method preserves an intermediate/undetermined region between the group means rather than forcing every score into one of two confident classes.

1 The paper defines a longitudinal atrophy-rate calculation based on change in `dem` divided by elapsed time.

1 Neerav co-authored the paper and, according to his resume and the portfolio evidence ledger, built the repeatable Python/OpenCV preprocessing and quantification pipeline.

What the evidence demonstrates

1 The method is interpretable because its prediction signal is an explicit ratio of observable pixel categories.

1. The crop experiment indicates that anatomically targeted preprocessing can improve this particular handcrafted signal.
1. The system can reduce repeated manual measurement effort in a research workflow, although the size of the time saving was not measured.
1. The `dem` score is useful as a research feature or baseline, not established as a clinical biomarker.

Scope boundaries

1. A unique patient count, subject-level train/test split, external validation cohort, or prospective clinical study.
1. Sensitivity, specificity, precision, recall, F1, confidence intervals, statistical significance, or a confusion matrix.
1. Accuracy against expert manual segmentations or a timing benchmark against expert tracing.
1. A reported longitudinal experiment with paired scans and observed atrophy-rate outcomes.
1. Generalization across scanners, acquisition protocols, institutions, demographic groups, or MRI modalities.
1. Safety or effectiveness for clinical diagnosis, prognosis, triage, or treatment selection.

These boundaries do not erase the contribution. They define it accurately: a published, interpretable computer-vision study that operationalized MRI atrophy cues, evaluated crop-dependent classification, and produced a reusable quantitative framework for further research.

Exact source map

1. **Full paper, local copy:** `/Users/neeravch/Downloads/Analyzing_Brain_Atrophy_in_Dementia_Using_Computer_Vision.pdf` (SHA-256 `b487524b46b0a681c85709f795f106653ef9ba5ec111cf0d868af62619f2a71b`).
1. Page 1: title, authors and affiliation; abstract; keywords; clinical and anatomical motivation; manual-segmentation problem; OpenCV research question; literature context.
1. Page 2: Methodology overview; ADNI and T1-weighted image description; Table I image counts and CDR grouping; Figures 1-3; implementation rationale; black, white, and grey pixel interpretation.
1. Page 3: Figure 4 anatomical comparison; Figure 5 pipeline; RGB thresholds; Equation 1 (`dem`); Equations 2-3 (group means/bounds); three-region classification rule.
1. Page 4: Equation 4 (atrophy rate); Results and Analysis; Figure 6 (`dem = 0.60` example); Figure 7 bounds and score plot; Table II crop accuracies; Conclusion; references 1-7.
1. Page 5: references 8-14.
1. **Canonical publication record:** IEEE DOI landing page, used for DOI, conference identity, authorship, conference dates/location, and Xplore publication date.
1. **Proceedings table of contents:** ICRITO 2022 table of contents, used for proceedings pagination 1234-1238 and neighboring-paper sequence.
1. **Portfolio evidence ledger:** `docs/CONTENT_SOURCE_LEDGER.md`, Brain atrophy publication section, used to corroborate the intended portfolio boundary and resume source.
1. **Resume corroboration:** `public/uploads/references/Resume_Neerav_Chaudhary.pdf`, used only for Neerav's individual implementation role: the repeatable Python/OpenCV MRI preprocessing and brain-atrophy quantification pipeline.
1. **Existing project record:** `content/projects/brain-atrophy-computer-vision.json`, used to identify current portfolio framing and to ensure the paper remains distinct from the later four-class TensorFlow/CNN ensemble notebook.
1. **Existing publication archive record:** `content/archive/alzheimers-disease-ieee-record.json`, used to cross-check the public citation and separation from the later classification study.